

第一章 C++基础（面试突击★★★★★）

适用对象：3-5年后端开发工程师 | **语言标准：**C++17 | **编译环境：**g++ -std=c++17

阅读建议：每个概念的面试题和代码示例是重中之重，建议先看面试题再看原理。

目录

1. [auto 类型推导](#)
2. [decltype 类型推导](#)
3. [nullptr 空指针](#)
4. [using 类型别名](#)
5. [constexpr 常量表达式](#)
6. [Lambda 表达式](#)
7. [移动语义 std::move](#)
8. [完美转发 std::forward](#)
9. [左值与右值 lvalue/rvalue](#)
10. [RAII 资源管理](#)
11. [智能指针](#)
12. [vector 动态数组](#)
13. [deque 双端队列](#)
14. [list 双向链表](#)
15. [map 有序映射](#)
16. [unordered_map 无序映射](#)
17. [set 有序集合](#)
18. [unordered_set 无序集合](#)
19. [string 字符串](#)
20. [模板基础](#)
21. [模板特化](#)
22. [偏特化](#)
23. [可变参数模板](#)
24. [std::thread](#)
25. [std::mutex](#)
26. [std::condition_variable](#)
27. [std::atomic](#)
28. [std::future](#)
29. [std::async](#)
30. [章节总结](#)

1. auto 类型推导

1.1 是什么（定义）

`auto` 是 C++11 引入的**类型占位符**，编译器在编译期根据初始化表达式自动推导变量的实际类型。C++14 扩展了 `auto` 用于函数返回类型推导，C++17 支持 structured binding 中的 `auto` 类型推导。

```
auto x = 42;           // int
auto y = 3.14;         // double
auto z = "hello";      // const char*
auto v = std::vector<int>{1,2,3}; // std::vector<int>
```

1.2 为什么会出现

- 1. **简化复杂类型书写**：迭代器声明从 `std::vector<int>::const_iterator it = v.begin()` 变为 `auto it = v.begin()`。
- 2. **泛型编程需求**：模板中依赖类型名不确定，`auto` 让编译器自动处理返回类型推导。
- 3. **减少类型不匹配错误**：避免手动写错类型导致的隐式转换和截断。
- 4. **支持 lambda 存储**：lambda 类型是编译器生成的匿名类型（闭包类），没有 `auto` 无法用变量存储。

1.3 底层原理

`auto` 的推导规则与**模板参数推导（template argument deduction）** 完全一致，只有一点例外：`auto` 可以推导 `std::initializer_list`，而模板参数推导不行。

推导规则核心（`auto` 会剥去引用和顶层 `const`）：

auto x = expr;

→ 模板推导，剥去引用和顶层const

auto& x = expr;

→ 保留引用

const auto& x = expr;

→ 保留底层const+引用

auto&& x = expr;

→ 转发引用（万能引用），保留一切

decltype(auto) x = expr;

→ 不剥任何东西，完全保留expr的类型

特殊情况：

auto x = {1,2,3};

→ std::initializer_list<int>

auto x{1};

→ C++17: int（直接初始化，非initializer_list）

1.4 优缺点

优点	缺点
简化冗长类型名，提升可读性	过度使用降低代码可读性（不知道变量类型）
减少类型不匹配错误	可能发生不期望的类型推导（如 <code>auto</code> 推导为值而非引用）
泛型编程必备	调试时类型不直观
编译器自动处理，零开销	<code>auto</code> 会剥去引用和cv限定符，可能产生意外拷贝

何时使用：迭代器、lambda、模板返回类型、范围for循环。

何时避免：需要明确类型语义的场合（如需要固定大小的int16_t而非int）、需要引用语义时（优先用 auto& 或 decltype(auto)）。

1.5 典型应用

```
// 1. 迭代器
std::map<std::string, std::vector<int>> m;
auto it = m.find("key"); // 替代 std::map<std::string, std::vector<int>>::iterator

// 2. Lambda
auto cmp = [](int a, int b) { return a > b; };

// 3. 范围for
std::vector<std::string> names;
for (const auto& name : names) { /* ... */ }

// 4. 结构化绑定 (C++17)
std::map<int, std::string> m;
for (auto&& [key, value] : m) { /* ... */ }

// 5. 泛型lambda (C++14)
auto generic_lambda = [](auto x, auto y) { return x + y; };

// 6. 尾置返回类型
template <typename T, typename U>
auto add(T t, U u) -> decltype(t + u) { return t + u; }
```

1.6 完整代码示例

```
#include <iostream>
#include <vector>
#include <map>
#include <type_traits>
#include <string>

int main() {
    // === 基本类型推导 ===
    auto i = 42; // int
    const auto ci = 42; // const int
    auto& ri = i; // int&
    const auto& cri = i; // const int&
    auto&& rri = 42; // int&& (转发引用绑定右值)
    auto&& rri2 = i; // int& (转发引用绑定左值)

    ri = 100; // i 变为 100

    // === 初始化列表 ===
    auto il = {1, 2, 3, 4}; // std::initializer_list<int>
    auto x{1}; // C++17: int (直接初始化)
```

```

// === 复杂类型简化 ===
std::map<std::string, std::vector<int>>> data;
data["scores"] = {95, 88, 92};
// 没有 auto: std::map<std::string, std::vector<int>>::iterator
auto it = data.find("scores");
if (it != data.end()) {
    for (const auto& score : it->second) {
        std::cout << score << " ";
    }
}
std::cout << std::endl;

// === Lambda ===
auto add = [](auto a, auto b) { return a + b; }; // C++14 泛型lambda
std::cout << add(3, 5) << std::endl; // 8
std::cout << add(3.14, 2.0) << std::endl; // 5.14

// === 结构化绑定 (C++17) ===
std::pair<int, std::string> p{1, "hello"};
auto [num, str] = p;
std::cout << num << ": " << str << std::endl;

// === decltype(auto) ===
int val = 10;
auto a = val; // int (剥去引用)
decltype(auto) da = val; // int (val是左值但非引用表达式)
decltype(auto) da2 = (val); // int& (加了括号变成左值表达式, decltype规则)

std::cout << "All tests passed!" << std::endl;
return 0;
}

```

1.7 面试高频问题（至少5道+标准答案）

Q1: auto 会推导为引用吗？

答案：不会。 `auto x = expr` 按值推导，剥去引用和顶层const。如果需要引用，必须显式写 `auto& x = expr`。而 `decltype(auto)` 会保留引用。

Q2: auto 和 decltype(auto) 有什么区别？

答案：auto 使用模板参数推导规则（剥去引用和顶层const）；decltype(auto) 使用 decltype 的推导规则（完全保留表达式的类型和值类别）。例如 `int x=0; int& rx=x; auto a=rx; // int; decltype(auto) b=rx; // int&`。

Q3: auto x = {1,2,3} 推导为什么类型？ auto x{1} 呢？

答案：`auto x = {1,2,3}` 推导为 `std::initializer_list<int>`。`auto x{1}` 在 C++17 中推导为 `int`（直接初始化，不再是 `initializer_list`）。这是 auto 特有的规则，模板参数推导不支持 `initializer_list`。

Q4: 范围for循环中 auto、auto&、const auto& 怎么选？

答案：`auto` 会拷贝每个元素，适用于小类型（int等）；`auto&` 允许修改元素且不拷贝，但不能绑定临时对象；`const auto&` 最通用，不拷贝且不能修改，可绑定临时对象。推荐默认使用 `const auto&`。

Q5: 为什么 auto 不能用作函数参数类型?

答案: C++20 之前, auto 不能用于普通函数参数 (只能用于lambda参数即泛型lambda)。因为这等价于未约束的模板, 编译器无法确定函数签名。C++20 的 abbreviated function templates 允许了这种写法, 它等价于隐式模板。

Q6: 什么是 auto 的类型退化 (decay) ?

答案: auto 推导时会进行类型退化 (decay), 类似于按值传参: 数组退化为指针 (`int[10]` $\rightarrow$ `int*`), 函数退化为函数指针, 剥去引用和顶层cv限定符。

1.8 延伸问题

- auto 与模板参数推导在哪些场景下结果不同?
- `vector<bool>` 的 `operator[]` 返回代理对象, `auto b = v[0]` 会发生什么?
- C++20 concept 与 auto 结合使用的场景?
- 如何用 `auto&&` 实现完美转发?

2. decltype 类型推导

2.1 是什么 (定义)

`decltype` 是 C++11 引入的关键字, 用于在编译期获取表达式或实体的**声明类型 (declared type)**。与 `auto` 不同, `decltype` 不剥去引用和 cv 限定符。

```
int x = 0;
decltype(x) y = 1;           // int
decltype((x)) z = x;         // int& (注意双括号)
```

2.2 为什么会出现

1. **泛型编程中推导返回类型:** 模板函数返回类型依赖于参数类型, 无法事先确定。
2. **声明与某表达式类型相同的变量:** 避免手动重复复杂的类型名。
3. **auto 的补充:** `auto` 会剥去引用, `decltype` 不会, 二者互补。

2.3 底层原理

`decltype` 的推导规则分两种情况:

规则1: 如果表达式 `e` 是一个不加括号的标识符 (`id-expression`)
或类成员访问表达式 → 推导为该标识符的声明类型
`decltype(x)` → `x`的声明类型 (如果是`int`, 就是`int`)

规则2: 如果表达式 `e` 是其他表达式 (加了括号、函数调用、运算等)
a) `e` 是左值 (`lvalue`) → `T&`
b) `e` 是右值 (`rvalue`) → `T&&`
c) `e` 是纯右值 (`prvalue`) → `T`

关键记忆点:

`decltype(x)` → 标识符本身的类型 (`int`)
`decltype((x))` → 左值表达式, 返回`int&` (因为`(x)`是左值表达式)

2.4 优缺点

优点	缺点
精确保留类型 (含引用和cv)	语法不如 <code>auto</code> 简洁
泛型编程中推导返回类型的利器	双括号陷阱 (<code>decltype((x))</code> 返回引用)
可推导函数/数组类型 (不会退化)	对于复杂表达式可能得到意料之外的类型

2.5 典型应用

```
// 1. 尾置返回类型 (trailing return type)
template <typename T1, typename T2>
auto add(T1 a, T2 b) -> decltype(a + b) { return a + b; }

// 2. 推导返回类型 (C++14后可用decltype(auto))
template <typename Container>
decltype(auto) getElement(Container& c, size_t idx) {
    return c[idx]; // 保留引用返回
}

// 3. 声明与已知表达式同类型的变量
std::vector<int> v;
decltype(v)::value_type val = 10; // int

// 4. 完美转发返回类型
template <typename F, typename... Args>
decltype(auto) invoke(F&& f, Args&&... args) {
    return std::forward<F>(f)(std::forward<Args>(args)...);
}
```

2.6 完整代码示例

```
#include <iostream>
#include <vector>
#include <type_traits>

template <typename T1, typename T2>
auto multiply(T1 a, T2 b) -> decltype(a * b) {
    return a * b;
}

template <typename Container>
decltype(auto) getAt(Container& c, size_t idx) {
    return c[idx]; // 保留引用语义
}

int main() {
    // === 基本decltype ===
    int x = 42;
    const int cx = 100;
    int& rx = x;

    decltype(x) a = 0; // int
    decltype(rx) b = x; // int&
    decltype(cx) c = 0; // const int

    static_assert(std::is_same_v<decltype(a), int>);
    static_assert(std::is_same_v<decltype(b), int&>);
    static_assert(std::is_same_v<decltype(c), const int>);

    // === 双括号陷阱 ===
    decltype(x) d1 = 10; // int
    decltype((x)) d2 = x; // int& ((x)是左值表达式)

    // === 数组不退化 ===
    int arr[5] = {1,2,3,4,5};
    decltype(arr) arr2 = {6,7,8,9,10}; // int[5], 不是int*
    // auto arr3 = arr; → int* (退化)

    // === 尾置返回类型 ===
    auto result = multiply(3, 4.5);
    std::cout << "multiply(3, 4.5) = " << result << std::endl;
    static_assert(std::is_same_v<decltype(result), double>);

    // === decltype(auto) 返回引用 ===
    std::vector<int> vec = {10, 20, 30};
    decltype(auto) elem = getAt(vec, 1);
    elem = 999; // 修改的是引用, vec[1]变为999
    std::cout << "vec[1] = " << vec[1] << std::endl;

    // === 表达式值类别推导 ===
    int&& rva1 = 42;
```

```

decltype(rval)      r1 = 10;    // int&&
decltype(std::move(r1)) r2 = 20; // int&& (xvalue→T&&)

// === 函数类型 ===
decltype(multiply<int, double>)* fp = multiply<int, double>;
std::cout << "fp(2, 3.0) = " << fp(2, 3.0) << std::endl;

std::cout << "All decltype tests passed!" << std::endl;
return 0;
}

```

2.7 面试高频问题（至少5道+标准答案）

Q1: decltype(x) 和 decltype(x) 有什么区别？

答案：这是最经典的陷阱题。decltype(x) 中 x 是不加括号的标识符，返回 x 的声明类型（如 int）。decltype((x)) 中 (x) 是一个表达式（左值表达式），所以返回 int&。面试官经常通过这个考察对值类别的理解。

Q2: auto 和 decltype 的核心区别是什么？

答案：auto 使用模板参数推导规则（剥去引用和顶层const，类型会decay）；decltype 直接获取表达式的声明类型（保留引用和cv，不decay）。auto 必须初始化，decltype 不需要。

Q3: decltype(auto) 是什么？什么时候用？

答案：C++14 引入，用于函数返回类型推导。它告诉编译器：用 decltype 的规则来推导返回值，而不是 auto 的规则。主要用于泛型代码中需要完美转发返回值的场景（保留引用语义）。

Q4: decltype 如何推导函数调用表达式？

答案：decltype(f(args)) 推导为函数的返回类型。注意：函数实际上不会被执行，decltype 是编译期行为，只做类型分析。

Q5: decltype 推导数组和函数时会退化吗？

答案：不会。这是 decltype 和 auto 的重要区别。int arr[10]; decltype(arr) a; 中 a 的类型是 int[10] 而非 int*。同理，函数类型也不会退化为函数指针。

2.8 延伸问题

- decltype 和 std::declval 如何配合使用？
- 如何在 C++11 中实现类似于 decltype(auto) 的效果？
- decltype 在 SFINAE 中的应用？
- auto 返回类型推导 + decltype 尾置返回类型的区别？

3. nullptr 空指针

3.1 是什么（定义）

`nullptr` 是 C++11 引入的关键字，类型为 `std::nullptr_t`，表示**空指针常量**。它是一个纯右值（prvalue），可以隐式转换为任意指针类型或成员指针类型，但**不能隐式转换为整数类型**。

```
int* p = nullptr;           // OK
void (*fp)() = nullptr;     // OK: 函数指针
int n = nullptr;            // 编译错误: 不能转为int
```

3.2 为什么会出现

- 1. **消除 NULL 的二义性**: `NULL` 在 C++ 中通常定义为 `0` 或 `0L`（整数），导致重载决议时匹配 `int` 而非指针。
- 2. **类型安全**: `nullptr` 有独立类型 `std::nullptr_t`，编译器可以区分指针和整数上下文。
- 3. **模板安全**: 在模板中传递 `NULL` 可能被推导为 `int`，而 `nullptr` 始终被推导为 `std::nullptr_t`。

3.3 底层原理

```
// nullptr 的简化实现等价于：
namespace std {
    using nullptr_t = decltype(nullptr);
    // nullptr_t 是一个独立类型，不是整数类型
}
// 编译器将其视为一个特殊的字面量，类似于 true/false
```

`nullptr` 是一种**编译期常量**，不是普通变量。编译器在遇到 `nullptr` 时直接将目标指针置零。
`sizeof(nullptr_t) == sizeof(void*)`。与 `NULL`（本质是整数0）相比，`nullptr` 有明确的指针语义。

3.4 优缺点

优点	缺点
类型安全，不会错误匹配整数重载	无（完全替代 NULL）
模板中类型推导正确	-
语义明确，代码意图清晰	-
可参与函数重载区分（ <code>nullptr_t</code> 参数）	-

3.5 典型应用

```
// 1. 指针初始化
int* p = nullptr;

// 2. 函数重载区分
void f(int)    { /* 整数版本 */ }
void f(void*) { /* 指针版本 */ }
```

```

f(0);          // 调用 f(int)
f(nullptr);    // 调用 f(void*) ← 明确意图

// 3. 条件判断
if (p == nullptr) { /* ... */ }
if (!p)           { /* ... */ } // 也合法

// 4. 模板中
template <typename T>
void g(T t) { /* ... */ }
g(0);           // T = int
g(nullptr);     // T = std::nullptr_t

```

3.6 完整代码示例

```

#include <iostream>
#include <type_traits>
#include <cstdint>

// 演示 NULL vs nullptr 的重载差异
void overloaded(int) {
    std::cout << "Called overloaded(int)" << std::endl;
}
void overloaded(void*) {
    std::cout << "Called overloaded(void*)" << std::endl;
}
void overloaded(std::nullptr_t) {
    std::cout << "Called overloaded(std::nullptr_t)" << std::endl;
}

// 模板中演示
template <typename T>
void deduce(T t) {
    if constexpr (std::is_same_v<T, std::nullptr_t>) {
        std::cout << "Deduced as std::nullptr_t" << std::endl;
    } else if constexpr (std::is_same_v<T, int>) {
        std::cout << "Deduced as int" << std::endl;
    }
}

int main() {
    // === 基本用法 ===
    int* p1 = nullptr;           // int*
    double* p2 = nullptr;        // double*
    void (*fp)() = nullptr;       // 函数指针

    // === nullptr_t 类型 ===
    std::nullptr_t np = nullptr;
    static_assert(sizeof(np) == sizeof(void*));

    // === 重载决议 ===
    std::cout << "=== Overload Resolution ===" << std::endl;
}

```

```

overloaded(0);          // 调用 overloaded(int)
overloaded(nullptr);    // 调用 overloaded(std::nullptr_t), 最匹配

// === 模板中的区别 ===
std::cout << "=== Template Deduction ===" << std::endl;
deduce(0);              // T = int
deduce(nullptr);        // T = std::nullptr_t

// === 条件判断 ===
if (p1 == nullptr) {
    std::cout << "p1 is null" << std::endl;
}
if (!p1) {
    std::cout << "p1 is also null (via ! operator)" << std::endl;
}
// if(p1) 等价于 if(p1 != nullptr)

std::cout << "All nullptr tests passed!" << std::endl;
return 0;
}

```

3.7 面试高频问题（至少5道+标准答案）

Q1: NULL 和 nullptr 的本质区别是什么？

答案：NULL 是宏，通常定义为 0 或 ((void*)0) (C中)，在 C++ 中是整数类型。nullptr 是关键字，类型为 std::nullptr_t，只能转换为指针类型不能转为整数。核心区别在于重载决议和模板推导中的行为不同。

Q2: nullptr 的类型是什么？大小是多少？

答案：std::nullptr_t，定义在 <cstddef> 中。sizeof(nullptr_t) 通常等于 sizeof(void*) (在 32 位系统为 4 字节，64 位为 8 字节)。

Q3: 为什么在模板中推荐用 nullptr 而不是 NULL？

答案：template<typename T> void f(T t) 中，f(NULL) 推导 T=int，而 f(nullptr) 推导 T=std::nullptr_t。使用 NULL 可能导致类型推导错误，进而引发非预期的行为。

Q4: if(p) 和 if(p != nullptr) 等价吗？

答案：等价。if(p) 会隐式调用 p != nullptr 做条件判断。两种写法都合法，但 if(p != nullptr) 意图更明确，推荐使用。

Q5: 可以用 delete nullptr 吗？

答案：可以，delete nullptr 是合法且安全的操作（等价于 delete 空指针，什么也不做）。但这在代码中很少有意义。

Q6: nullptr 可以赋给成员函数指针吗？

答案：可以。nullptr 可以隐式转换为任意指针类型，包括成员函数指针：void (MyClass::*mfp)() = nullptr;。

3.8 延伸问题

- C++11 之前的代码库如何平滑迁移到 nullptr?
- nullptr_t 可以用于模板元编程吗?
- 为什么 `int x = nullptr;` 编译失败, 其设计动机是什么?
- 智能指针 (shared_ptr/unique_ptr) 与 nullptr 的比较机制?

4. using 类型别名

4.1 是什么 (定义)

`using` 在 C++11 中获得了新能力: 可以声明**类型别名 (type alias)** 和**别名模板 (alias template)**, 作为 `typedef` 的现代替代。

```
// 类型别名
using StringVec = std::vector<std::string>;

// 别名模板 (typedef 做不到)
template<typename T>
using Vec = std::vector<T>;

// 函数指针
using FuncPtr = void (*)(int, double);
```

4.2 为什么会出现

1. **typedef 语法晦涩**: 函数指针的 typedef 写法极不直观 (类型名在中间位置)。
2. **typedef 不支持模板**: 无法用 typedef 创建带模板参数的别名 (需要用 struct 内嵌 typedef 迂回实现)。
3. **可读性**: `using A = B;` 符合现代赋值思维, 比 `typedef B A;` 更直观。

4.3 底层原理

`using` 别名在编译期**完全等效**于被别名类型, 不产生新的类型。编译器用符号表将别名替换为原始类型, 无运行时开销。

```
// 底层等价性
using MyInt = int;
MyInt x = 5;    // 编译器直接当作 int 处理, 生成的机器码完全相同

// 别名模板展开示例
template<typename T>
using MyVec = std::vector<T>;
MyVec<int> v;   // 编译器展开为 std::vector<int>
```

4.4 优缺点

方面	using	typedef
基本类型别名	<code>using A = B;</code>	<code>typedef B A;</code>
函数指针	<code>using F = void(*) (int);</code>	<code>typedef void(*F)(int);</code>
模板别名	支持	不支持(需要嵌套struct hack)
可读性	好 (类型在左边)	差 (类型名嵌在中间)

4.5 典型应用

```
// 1. 简化容器类型
template<typename K, typename V>
using HashMap = std::unordered_map<K, V>;

// 2. 函数指针
using Callback = std::function<void(int, const std::string&)>;

// 3. 跨平台类型定义
#ifdef _WIN64
using SocketHandle = uint64_t;
#else
using SocketHandle = int;
#endif

// 4. 弱化模板签名
template<typename T>
using Ptr = std::shared_ptr<T>;
Ptr<MyClass> obj = std::make_shared<MyClass>();
```

4.6 完整代码示例

```
#include <iostream>
#include <vector>
#include <map>
#include <memory>
#include <functional>
#include <type_traits>

// === 1. 基本类型别名 ===
using StringVec = std::vector<std::string>;
using IntMap = std::map<int, std::string>;

// === 2. 别名模板 (typedef做不到) ===
template<typename T>
using Ptr = std::shared_ptr<T>;

template<typename K, typename V>
```

```

using Dict = std::map<K, V>;

// === 3. 函数指针 ===
using Handler = void(*)(int, const char*);

// === 4. 复杂模板简化 ===
template<typename T>
using VecOfVec = std::vector<std::vector<T>>>;

// === typedef 等效写法对比 ===
typedef void(*OldHandler)(int, const char*); // 类型名OldHandler在中间

int main() {
    // using 别名
    StringVec names = {"Alice", "Bob", "Charlie"};
    IntMap scores = {{1, "A"}, {2, "B"}, {3, "C"}};

    // 别名模板
    Ptr<int> pi = std::make_shared<int>(42);
    Dict<std::string, double> prices = {{ "apple", 1.5}, {"banana", 2.0}};

    // 验证类型一致性
    static_assert(std::is_same_v<StringVec, std::vector<std::string>>);
    static_assert(std::is_same_v<Ptr<int>, std::shared_ptr<int>>);

    // 函数指针使用
    Handler h = [](int code, const char* msg) {
        std::cout << "Code: " << code << ", Msg: " << msg << std::endl;
    };
    h(200, "OK");

    // VecOfVec
    VecOfVec<int> matrix = {{1,2,3}, {4,5,6}, {7,8,9}};
    std::cout << "matrix[1][1] = " << matrix[1][1] << std::endl;

    std::cout << "All using tests passed!" << std::endl;
    return 0;
}

```

4.7 面试高频问题（至少5道+标准答案）

Q1: using 和 typedef 有什么区别？

答案：(1) using 支持别名模板，typedef 不支持。(2) 语法上 using 更直观（using A=B vs typedef B A）。(3) using 是 C++11 推荐写法，尤其是函数指针和模板别名场景。两者在基本类型别名上功能等价。

Q2: 什么时候 using 能做的事 typedef 做不到？

答案：创建别名模板（alias template）。`template<typename T> using Vec = std::vector<T>;` 无法用 typedef 实现。C++98/03 需要用 struct 嵌套 typedef 的迂回方式模拟。

Q3: using 别名会产生新类型吗？

答案：不会。using 只是给已有类型起了一个新名字，编译器在符号解析阶段将别名展开为原类型。

`std::is_same_v<MyInt, int>` 为 true。这与 enum class 不同（enum class 创建的是真正的强类型）。

Q4: typedef 在 C++17/20 中是否已过时？

答案：没有正式过时（deprecated），但 C++ Core Guidelines 推荐优先使用 using。除了兼容旧代码和少数习惯用法外，新项目应优先用 using。

Q5: 如何用 typedef 实现类似别名模板的效果？

答案：使用 trait class 内嵌 typedef：

```
template<typename T>
struct VecTrait {
    typedef std::vector<T> type;
};
VecTrait<int>::type v; // 等价于 std::vector<int>
```

显然不如 `template<typename T> using Vec = std::vector<T>; Vec<int> v;` 简洁。

4.8 延伸问题

- using 声明（`using std::cout;`）和 using 别名有什么区别？
- using 别名可以参与 SFINAE 吗？
- 头文件中如何正确管理 using 别名的作用域？
- C++20 concept 与 using 别名模板如何结合？

5. constexpr 常量表达式

5.1 是什么（定义）

`constexpr` 是 C++11 引入的关键字，用于声明**在编译期即可求值的表达式或函数**。C++14 放宽了 `constexpr` 函数的限制（允许局部变量、循环等），C++17 进一步强化（lambda 可以是 `constexpr`），C++20 引入了 `constexpr` 和 `constinit`。

```
constexpr int square(int n) { return n * n; }
constexpr int val = square(10); // 编译期计算, val = 100
int arr[val];                  // 合法: val 是编译期常量
```

5.2 为什么会出现

1. **性能优化：**将计算从运行时移到编译期，消除运行时开销。
2. **类型安全：**用编译期常量替代宏（如 `#define MAX 100`），支持类型检查和命名空间。
3. **模板元编程替代：**用普通函数语法替代复杂的模板元编程写法。
4. **数组大小等编译期需求：**C++ 中数组大小必须是编译期常量。

5.3 底层原理

编译器在编译阶段对 constexpr 函数进行**解释执行 (compile-time evaluation)**。当所有参数都是编译期已知的常量时，整个函数调用在编译期被求值，结果直接嵌入二进制代码。

编译期求值的条件：

1. 函数标记为 constexpr

2. 所有参数是编译期常量

3. 函数体内所有操作在编译期可执行（不涉及运行时操作）

const vs constexpr：

const → "运行时不修改"（可能是运行期初始化）

constexpr → "编译期确定"（一定是编译期初始化的）

constexpr 隐含 const，反之不成立

5.4 优缺点

优点	缺点
编译期计算，零运行时开销	C++11 函数体内只能有一条 return 语句
类型安全，替代宏	不能包含任何运行时操作（I/O、动态分配等）
可用于数组大小、模板参数等处	过度使用可能导致编译时间变长、二进制膨胀
C++14/17 大幅放宽限制	调试编译期代码较困难

5.5 典型应用

```
// 1. 编译期常量
constexpr int MAX_SIZE = 1024;
char buffer[MAX_SIZE]; // 合法：MAX_SIZE是编译期常量

// 2. constexpr 函数
constexpr int factorial(int n) {
    return n <= 1 ? 1 : n * factorial(n - 1);
}

// 3. constexpr 类
struct Point {
    double x, y;
    constexpr Point(double xx, double yy) : x(xx), y(yy) {}
    constexpr double dist() const { return x*x + y*y; }
};

// 4. constexpr if (C++17)
template<typename T>
auto getValue(T t) {
    if constexpr (std::is_pointer_v<T>)
        return *t;
    else
```



```
        return t;
    }
}
```

5.6 完整代码示例

```
#include <iostream>
#include <array>
#include <type_traits>

// === constexpr 函数 (C++14风格) ===
constexpr int factorial(int n) {
    int result = 1;
    for (int i = 2; i <= n; ++i)
        result *= i;
    return result;
}

// === constexpr 类 ===
class Point {
public:
    constexpr Point(double x, double y) : x_(x), y_(y) {}
    constexpr double getX() const { return x_; }
    constexpr double getY() const { return y_; }
    constexpr double distance(const Point& other) const {
        double dx = x_ - other.x_;
        double dy = y_ - other.y_;
        return dx * dx + dy * dy;
    }
private:
    double x_, y_;
};

// === constexpr if (C++17) ===
template<typename T>
auto processValue(T&& val) {
    if constexpr (std::is_pointer_v<std::remove_reference_t<T>>()) {
        return *val;        // 指针解引用
    } else if constexpr (std::is_integral_v<std::remove_reference_t<T>>()) {
        return val * 2;      // 整数加倍
    } else {
        return val;          // 其他类型直接返回
    }
}

// === constexpr 变量 ===
constexpr int ARR_SIZE = factorial(5); // 编译期120

int main() {
    // 编译期求值
    constexpr int fact5 = factorial(5);
    std::cout << "factorial(5) = " << fact5 << std::endl;
    static_assert(fact5 == 120, "factorial(5) should be 120");
}
```

```

// 编译期数组大小
std::array<int, factorial(4)> arr = {}; // 大小为24
std::cout << "array size = " << arr.size() << std::endl;

// 运行时也可以调用 constexpr 函数
int n = 6;
std::cout << "factorial(" << n << ") = " << factorial(n) << std::endl;

// constexpr 类
constexpr Point p1(0, 0);
constexpr Point p2(3, 4);
constexpr double dist = p1.distance(p2);
std::cout << "distance squared = " << dist << std::endl;
static_assert(dist == 25.0, "3^2 + 4^2 = 25");

// constexpr if
int val = 10;
int* ptr = &val;
std::cout << "processValue(ptr) = " << processValue(ptr) << std::endl; // 10
std::cout << "processValue(val) = " << processValue(val) << std::endl; // 20

std::cout << "All constexpr tests passed!" << std::endl;
return 0;
}

```

5.7 面试高频问题（至少5道+标准答案）

Q1: const 和 constexpr 的区别？

答案：`const` 表示"运行时不修改变量"，但不保证编译期可知（如 `const int x = getValue()` 是运行时确定的）。`constexpr` 强保证"编译期可知"，所有 `constexpr` 隐含 `const`，但 `const` 不隐含 `constexpr`。`constexpr` 变量必须用编译期常量初始化。

Q2: constexpr 函数能在运行时调用吗？

答案：可以。`constexpr` 函数既可以编译期执行（当所有参数是编译期常量），也可以运行时执行（当任何参数是运行时变量）。这种"双模"能力是 `constexpr` 函数的核心优势。

Q3: 为什么 C++11 的 constexpr 函数只能有一条 return 语句？

答案：C++11 标准制定时保守起见，限制 `constexpr` 函数为单一表达式以避免实现复杂度。C++14 放宽限制，允许局部变量、循环、条件分支等。现在应使用 C++14 及以上的标准。

Q4: constexpr if (C++17) 和普通 if 有什么区别？

答案：`if constexpr` 在编译期评估条件，不满足条件的分支不会被实例化（编译）。这实现了编译期条件化代码生成，使得一个模板函数内可以有对不同类型的完全不同的处理逻辑。普通 `if` 需要两个分支对给定类型都是合法的。

Q5: constexpr 构造函数的用途？

答案：使得类的对象可以作为 `constexpr` 变量，在编译期初始化。这样，包含这些对象的容器（如 `std::array`）或数据可以在编译期构建，嵌入到只读数据段中，实现零运行时初始化开销。

5.8 延伸问题

- C++20 的 `constexpr` (立即函数) 和 `constexpr` 分别解决什么问题?
- `constexpr` 与 `inline` 的关系?
- 大数组用 `constexpr` 初始化会导致二进制膨胀, 如何平衡?
- `constexpr` 可以替代模板元编程吗? 各自的适用场景?

6. Lambda 表达式

6.1 是什么 (定义)

Lambda 表达式是 C++11 引入的**匿名函数对象 (闭包)**, 可以在代码中内联定义一个可调用对象。编译器自动生成一个唯一的匿名类 (闭包类), 并实例化一个对象。

```
// 基本语法
[capture](parameters) mutable -> return_type { body }

// 最简形式
auto f = []{ return 42; };

// 完整形式
auto g = [&x](int a) mutable -> int { x += a; return x; };
```

6.2 为什么会出现

1. **简化临时函数对象**: 替代 STL 中繁琐的仿函数 (functor) 写法。
2. **就地定义回调**: 不需要在远处定义函数或仿函数类, 代码更紧凑。
3. **捕获上下文**: 能直接访问局部变量, 比函数指针强大。
4. **函数式编程支持**: 配合 STL 算法实现 map/filter/reduce 模式。

6.3 底层原理

编译器将 lambda 转换为一个**匿名仿函数类**, 捕获列表决定成员变量的类型和初始化方式:

```
// Lambda: [x, &y](int a) { return x + y + a; }
// 编译器生成的等价代码:
class __lambda_123 {
    int x;           // 值捕获 -> 成员变量副本
    int& y;          // 引用捕获 -> 引用成员
public:
    __lambda_123(int _x, int& _y) : x(_x), y(_y) {}
    auto operator()(int a) const { return x + y + a; }
};
```

捕获方式的内存布局:

- `[]` → 空类 (大小通常为1字节)
- `[x]` → 类包含 `T` 成员
- `[&x]` → 类包含 `T&` 成员

[=]	→ 类包含所有捕获变量的副本
[&]	→ 类包含所有捕获变量的引用
[a, &b]	→ a的值副本 + b的引用
[*this]	→ C++17: 捕获当前对象的副本

捕获时机: 值捕获发生在 lambda 定义时（构造闭包对象时），而非调用时。这是常见陷阱。

6.4 优缺点

优点	缺点
代码紧凑，就地定义	过度使用降低可读性
支持上下文捕获	值捕获可能导致悬空引用（捕获指针/引用后原对象销毁）
可与 STL/异步无缝配合	每个 lambda 是独立类型，不能直接放入容器（需要 std::function 包装）
C++14 起支持泛型 lambda	递归 lambda 写法较复杂
零开销（编译器内联优化）	大型捕获列表可能增加闭包对象大小

6.5 典型应用

```
// 1. STL 算法
std::sort(v.begin(), v.end(), [](int a, int b) { return a > b; });

// 2. 异步回调
std::async(std::launch::async, []{ doWork(); });

// 3. 条件查找
auto it = std::find_if(v.begin(), v.end(), [threshold](int x) {
    return x > threshold;
});

// 4. 自定义删除器
auto deleter = [](FILE* f) { if(f) fclose(f); };
std::unique_ptr<FILE, decltype(deleter)> fp(fopen("a.txt", "r"), deleter);

// 5. C++14 泛型 lambda
auto add = [](auto a, auto b) { return a + b; };

// 6. C++17 constexpr lambda
constexpr auto sq = [](int n) { return n * n; };
```

6.6 完整代码示例

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <functional>
#include <memory>
```

```

int main() {
    // === 基本lambda ===
    auto hello = []() { std::cout << "Hello, Lambda!" << std::endl; };
    hello();

    // === 值捕获 ===
    int x = 10;
    auto byValue = [x]() { return x * 2; };
    std::cout << "byValue: " << byValue() << std::endl; // 20

    // === 引用捕获 ===
    auto byRef = [&x]() { x += 5; };
    byRef();
    std::cout << "x after byRef: " << x << std::endl; // 15

    // === 可变lambda (mutable允许修改值捕获的副本) ===
    auto mutableLambda = [x]() mutable {
        x += 1; // 修改的是副本, 不影响外部x
        return x;
    };
    std::cout << "mutable: " << mutableLambda() << std::endl; // 16
    std::cout << "original x: " << x << std::endl; // 15

    // === 泛型lambda (C++14) ===
    auto genericAdd = [](auto a, auto b) { return a + b; };
    std::cout << "genericAdd(3, 4.5): " << genericAdd(3, 4.5) << std::endl;
    std::cout << "genericAdd(string,string): "
        << genericAdd(std::string("Hello "), std::string("World")) << std::endl;

    // === STL排序 ===
    std::vector<int> numbers = {5, 2, 8, 1, 9, 3};
    // 降序排序
    std::sort(numbers.begin(), numbers.end(),
        [](int a, int b) { return a > b; });
    std::cout << "Sorted desc: ";
    for (int n : numbers) std::cout << n << " ";
    std::cout << std::endl;

    // === find_if ===
    int threshold = 5;
    auto it = std::find_if(numbers.begin(), numbers.end(),
        [threshold](int n) { return n < threshold; });
    if (it != numbers.end())
        std::cout << "First < " << threshold << ": " << *it << std::endl;

    // === lambda 作为 std::function ===
    std::function<int(int, int)> calculator = [](int a, int b) { return a * b; };
    std::cout << "calculator(6, 7): " << calculator(6, 7) << std::endl;

    // === C++14 初始化捕获 (init capture) ===
    auto ptrLambda = [p = std::make_unique<int>(42)]() { return *p; };
    std::cout << "ptrLambda: " << ptrLambda() << std::endl;
}

```

```
// === C++17 constexpr lambda ===
constexpr auto square = [](int n) { return n * n; };
constexpr int sq25 = square(5);
std::cout << "square(5): " << sq25 << std::endl;

std::cout << "All lambda tests passed!" << std::endl;
return 0;
}
```

6.7 面试高频问题（至少5道+标准答案）

Q1: lambda 表达式的底层实现原理是什么？

答案：编译器为每个 lambda 生成一个唯一的匿名仿函数类（闭包类型），捕获变量成为该类的成员变量，lambda 体成为 `operator()` 的实现。这是纯编译期行为，零运行时开销。没有捕获的 lambda 可以隐式转换为函数指针。

Q2: 值捕获和引用捕获的区别？什么时候用哪个？

答案：值捕获 `[x]` 在闭包构造时拷贝变量，之后对原变量的修改不影响闭包内的值。引用捕获 `[&x]` 存储变量的引用，之后可以通过闭包修改原变量。当 lambda 生命周期超过被捕获变量时，必须用值捕获（或确保变量存活），否则会产生悬空引用。大对象建议引用捕获避免拷贝。

Q3: mutable 关键字在 lambda 中的作用？

答案：默认情况下，lambda 的 `operator()` 是 const 的，不能修改值捕获的成员变量。加上 `mutable` 后，`operator()` 变为非 const，允许修改值捕获的副本。但修改的只是闭包内的副本，不影响原外部变量。

Q4: lambda 可以转换为函数指针吗？什么条件？

答案：只有没有捕获任何变量的 lambda（空捕获列表 `[]`）可以隐式转换为函数指针。有捕获的 lambda 必须用 `std::function` 包装。原因是捕获变量意味着闭包有状态，而函数指针是无状态的。

Q5: C++14 的泛型 lambda 和初始化捕获分别是什么？

答案：泛型 lambda：参数类型用 `auto`，`[](auto a, auto b) { return a + b; }`。初始化捕获 (init capture)：`[p = std::make_unique<int>(42)]` 允许在捕获列表中初始化任意类型的变量，解决了 C++11 中无法值捕获 move-only 类型的问题。

Q6: 如何用 lambda 实现递归？

答案：C++14 后可以用泛型 lambda + `std::function` 或 Y 组合子实现。最简单的做法是：

```
std::function<int(int)> fib = [&fib](int n) {
    return n <= 1 ? n : fib(n-1) + fib(n-2);
};
```

注意必须用 `std::function`（不能用 `auto`），且捕获 `fib` 本身需要引用捕获。

6.8 延伸问题

- lambda 捕获 `*this` (C++17) 的使用场景和价值？
- `[=]` 和 `[&]` 的隐含风险？哪个更安全？
- C++20 的模板 lambda (`[]<typename T>(T t){}`) 与泛型 lambda 的区别？
- lambda 与 `std::bind` 相比的优势？

7. 移动语义 `std::move`

7.1 是什么（定义）

移动语义是 C++11 引入的核心特性，允许将资源从一个对象**转移**到另一个对象，而非深拷贝。`std::move` 本身只是一个类型转换（将左值强制转为右值引用），不移动任何东西。真正的移动操作由**移动构造函数**和**移动赋值运算符**完成。

```
std::string s1 = "hello";
std::string s2 = std::move(s1); // 移动: s1的资源转移到s2
// s1 现在处于"有效但未指定"的状态（通常是空字符串）
```

7.2 为什么会出现

1. **消除不必要的深拷贝**：临时对象、返回值等场景中拷贝开销巨大。
2. **支持 move-only 类型**：`unique_ptr`、`thread`、`fstream` 等资源独占类型需要所有权转移语义。
3. **性能优化**：容器中元素重分配时，移动比拷贝高效得多（ $O(1)$ vs $O(n)$ ）。
4. **完美转发的基础**：移动语义是右值引用体系的核心组成部分。

7.3 底层原理

```
std::move 的实现（简化版）：
template<typename T>
typename std::remove_reference<T>::type&& move(T&& t) noexcept {
    return static_cast<typename std::remove_reference<T>::type&&>(t);
}
// 本质：无条件将参数转换为右值引用
```

移动构造函数的工作流程：

1. 接收一个右值引用参数 `T&& other`
2. "窃取" `other` 的内部资源指针
3. 将 `other` 的资源指针置为空（使其可安全析构）

图示（以 `std::string` 为例）：

移动前：

s1 -> [H][e][l][l][o]\0 （堆内存）
s2 -> （未初始化）

移动后：

s1 -> nullptr （被"掏空"）
s2 -> [H][e][l][l][o]\0 （接管了s1的资源）

7.4 优缺点

优点	缺点
避免深拷贝，大幅提升性能	移动后源对象状态不确定，容易误用
支持 move-only 类型的值语义	过度使用 std::move 可能抑制编译器优化（RVO/NRVO）
配合容器可实现高效的resize操作	移动构造函数必须标记 noexcept 才能被某些容器使用
零额外开销	对没有堆资源的对象（如基本类型、POD），move 等于 copy

何时使用：返回局部变量时（return std::move 反而抑制 RVO）、将对象转入容器、所有权转移、unique_ptr 传递。

何时避免：return 局部对象（依赖 RVO）、对 const 对象（move 退化为 copy）、不需要转移所有权的场景。

7.5 典型应用

```
// 1. 容器高效操作
std::vector<std::string> v;
std::string s = "hello";
v.push_back(std::move(s));    // 移动而非拷贝

// 2. unique_ptr 所有权转移
std::unique_ptr<int> p1 = std::make_unique<int>(42);
std::unique_ptr<int> p2 = std::move(p1);    // p1变为nullptr

// 3. 交换操作
template<typename T>
void swap(T& a, T& b) {
    T tmp = std::move(a);
    a = std::move(b);
    b = std::move(tmp);
}

// 4. 工厂函数返回大对象
std::vector<int> createLargeVector() {
    std::vector<int> v(1000000);
    return v;    // 编译器自动使用移动（或RVO），不要写 return std::move(v)
}
```

7.6 完整代码示例

```
#include <iostream>
#include <vector>
#include <string>
#include <memory>
#include <chrono>

// 模拟一个带堆资源的类
class Buffer {
```



```

public:
    explicit Buffer(size_t size) : size_(size), data_(new int[size]) {
        std::cout << " Buffer(size_t) - allocate " << size_ << " ints" << std::endl;
    }

    // 拷贝构造（深拷贝）
    Buffer(const Buffer& other) : size_(other.size_), data_(new int[other.size_]) {
        std::copy(other.data_, other.data_ + size_, data_);
        std::cout << " Buffer(const&) - deep copy " << size_ << " ints" << std::endl;
    }

    // 移动构造（资源窃取）
    Buffer(Buffer&& other) noexcept
        : size_(other.size_), data_(other.data_) {
        other.size_ = 0;
        other.data_ = nullptr; // 关键：置空源对象的指针
        std::cout << " Buffer(&&) - move " << size_ << " ints" << std::endl;
    }

    // 移动赋值
    Buffer& operator=(Buffer&& other) noexcept {
        if (this != &other) {
            delete[] data_;
            size_ = other.size_;
            data_ = other.data_;
            other.size_ = 0;
            other.data_ = nullptr;
        }
        std::cout << " operator=(&&) - move assign" << std::endl;
        return *this;
    }

    ~Buffer() {
        delete[] data_;
        if (data_) std::cout << " ~Buffer() - free " << size_ << " ints" << std::endl;
    }

    size_t size() const { return size_; }

private:
    size_t size_;
    int* data_;
};

// 工厂函数
Buffer createBuffer() {
    Buffer b(100);
    return b; // 编译器使用 RVO 或移动，不要写 return std::move(b)
}

int main() {
    std::cout << "=== 构造 ===" << std::endl;
    Buffer b1(1000);

```

```

std::cout << "\n=== 移动构造 ===" << std::endl;
Buffer b2 = std::move(b1); // b1的资源转移到b2
std::cout << "b1.size() = " << b1.size() << std::endl; // 0
std::cout << "b2.size() = " << b2.size() << std::endl; // 1000

std::cout << "\n=== 容器移动 ===" << std::endl;
std::vector<Buffer> buffers;
buffers.push_back(Buffer(500)); // 临时对象，自动移动
buffers.push_back(std::move(b2)); // b2转移进容器
std::cout << "b2.size() after move: " << b2.size() << std::endl; // 0

std::cout << "\n=== unique_ptr 移动 ===" << std::endl;
auto p1 = std::make_unique<int>(42);
auto p2 = std::move(p1); // 所有权转移
// p1 现在是 nullptr
if (!p1) std::cout << "p1 is nullptr after move" << std::endl;
std::cout << "*p2 = " << *p2 << std::endl;

std::cout << "\n=== RVO 演示 ===" << std::endl;
Buffer b3 = createBuffer(); // RVO，不调用移动/拷贝构造
std::cout << "b3.size() = " << b3.size() << std::endl;

std::cout << "\n=== 析构（程序结束） ===" << std::endl;
return 0;
}

```

7.7 面试高频问题（至少5道+标准答案）

Q1: std::move 到底做了什么？真的"移动"了吗？

答案：`std::move` 什么也不移动。它只是一个类型转换：将左值无条件转换为右值引用（`static_cast<T&&>`）。真正的移动操作由移动构造函数或移动赋值运算符执行。`std::move` 更像是"允许移动"的标记。

Q2: 被 move 后的对象还能用吗？

答案：处于"有效但未指定"的状态（valid but unspecified）。可以安全地析构或重新赋值，但不应对其内容做任何假设（如 `s.empty()` 结果不确定）。通常用于：赋值新值、`clear()` 重置、或直接析构。

Q3: 为什么移动构造函数要加 noexcept？不加有什么后果？

答案：`std::vector` 在扩容时，如果移动构造函数是 `noexcept`，会选择移动元素；否则会退化为拷贝（保证异常安全）。因为如果移动中途抛异常，已移动的元素无法恢复。不加 `noexcept` 会损失大量性能。

Q4: return 局部变量时，应该写 return std::move(x) 吗？

答案：绝对不要！ 编译器对 `return` 局部变量有特殊优化（RVO/NRVO），会直接在调用者的内存上构造对象，比移动更高效。写了 `std::move` 反而抑制了 RVO，强制使用移动（甚至拷贝）。只有返回非局部变量或函数参数时才需要 `std::move`。

Q5: 什么类型适合定义移动语义？

答案：持有堆资源或独占资源的类适合定义移动语义：`std::string`、`std::vector`、`std::unique_ptr`、文件句柄类、socket 封装类、大缓冲区类。POD 类型（如 `int`、`Point`）移动等于拷贝，不需要定义移动语义。

Q6: 如何判断一个类是否可移动？

答案：使用 `std::is_move_constructible<T>` 和 `std::is_move_assignable<T>`。如果一个类定义了移动构造函数/移动赋值运算符（或编译器自动生成），且移动操作是 `noexcept` 的（通过 `std::is_nothrow_move_constructible` 判断），则它是有效可移动的。

7.8 延伸问题

- 移动语义和 RVO/NRVO 的区别与优先级？
- 编译器何时自动生成移动构造函数？Rule of Five 是什么？
- 如何实现一个支持移动语义的容器类？
- 移动语义在多线程环境下的安全性考量？

8. 完美转发 `std::forward`

8.1 是什么（定义）

`std::forward` 是 C++11 引入的**条件类型转换**工具，配合**转发引用（万能引用）**使用，能够保持参数的值类别（lvalue/rvalue），实现参数的“完美转发”——即转发后的参数保持原来的左值/右值属性。

```
template<typename T>
void wrapper(T&& arg) {
    // arg 在函数内部是一个左值（有名字的右值引用是左值）
    target(std::forward<T>(arg)); // 完美转发：保持arg原来的值类别
}
```

8.2 为什么会出现

1. **泛型代码中保持值类别：**模板函数接收参数后，所有具名参数都变成左值，直接转发会丢失右值语义。
2. **避免重复编写重载：**一个模板函数需要同时处理左值和右值参数，不需要分别编写 `const T&` 和 `T&&` 两个版本。
3. **性能优化：**保持右值特性以便下游函数可以使用移动语义。
4. **通用工厂函数：**`std::make_shared`、`std::make_unique`、`emplace_back` 等都需要完美转发。

8.3 底层原理

转发引用（`T&&` 在模板推导上下文中）的类型推导规则：

```
wrapper(x)      <- x是左值 -> T 推导为 int&, T&& 展开为 int& && -> int&
wrapper(42)     <- 42是右值 -> T 推导为 int, T&& 展开为 int&&
```

引用折叠规则：

```
T& &    -> T&
T& &&   -> T&
```

```
T&& &    -> T&
T&& &&    -> T&&    （只有全是&&才得到&&）
```

std::forward 的实现（简化版）：

```
template<typename T>
T&& forward(typename std::remove_reference<T>::type& t) noexcept {
    return static_cast<T&&>(t);
}
// 如果 T=int, 返回 int&&
// 如果 T=int&, 引用折叠: int& && -> int&, 返回 int&
```

本质：std::move 是无条件转为右值，std::forward 是条件转换（右值的转右值，左值的保持左值）

8.4 优缺点

优点	缺点
一个模板函数处理所有值类别	只能用于模板上下文（需要推导）
保持移动语义，避免不必要拷贝	语法稍显复杂
消除重复代码	错误使用（如对非转发引用用forward）会导致迷惑行为
emplace 系列函数的基础	增加编译时间（模板实例化）

8.5 典型应用

```
// 1. 工厂函数
template<typename T, typename... Args>
std::unique_ptr<T> make_unique(Args&&... args) {
    return std::unique_ptr<T>(new T(std::forward<Args>(args)...));
}

// 2. emplace_back
std::vector<std::string> v;
v.emplace_back("hello"); // 内部用 forward 转发参数给 string 构造函数

// 3. 通用包装器
template<typename F, typename... Args>
decltype(auto) invoke(F&& f, Args&&... args) {
    return std::forward<F>(f)(std::forward<Args>(args)...);
}

// 4. 代理/装饰器模式
template<typename T>
class Proxy {
    T obj;
public:
    template<typename U>
    explicit Proxy(U&& u) : obj(std::forward<U>(u)) {}
};
```

8.6 完整代码示例

```
#include <iostream>
#include <utility>
#include <memory>
#include <string>
#include <vector>

// 接收者：区分左值和右值
void process(const std::string& s) {
    std::cout << " process(const&): " << s << std::endl;
}
void process(std::string&& s) {
    std::cout << " process(&&): " << std::move(s) << std::endl;
}

// 完美转发包装器
template<typename T>
void forwarder(T&& arg) {
    std::cout << "forwarder called, forwarding..." << std::endl;
    process(std::forward<T>(arg)); // 完美转发：保持值类别
}

// 错误示例：不用 forward
template<typename T>
void noForward(T&& arg) {
    std::cout << "noForward called, passing arg directly..." << std::endl;
    process(arg); // arg 是左值！始终调用 process(const&)
}

// 完美转发的工厂函数
template<typename T, typename... Args>
std::unique_ptr<T> my_make_unique(Args&&... args) {
    return std::unique_ptr<T>(new T(std::forward<Args>(args)...));
}

// emplace 模式示例
template<typename Container, typename... Args>
void emplaceBack(Container& c, Args&&... args) {
    c.emplace_back(std::forward<Args>(args)...);
}

class MyClass {
public:
    std::string name;
    int value;

    MyClass(const std::string& n, int v) : name(n), value(v) {
        std::cout << " MyClass(const&, int)" << std::endl;
    }
    MyClass(std::string&& n, int v) : name(std::move(n)), value(v) {
        std::cout << " MyClass(&&, int)" << std::endl;
    }
};
```

```

    }
};

int main() {
    std::cout << "=== 完美转发 ===" << std::endl;
    std::string s = "hello";
    forwarder(s);           // s是左值 -> process(const&)
    forwarder(std::string("world")); // 临时对象是右值 -> process(&&)

    std::cout << "\n=== 不用forward的问题 ===" << std::endl;
    noForward(std::string("temp")); // 始终调用 process(const&)

    std::cout << "\n=== 工厂函数 ===" << std::endl;
    auto p1 = my_make_unique<MyClass>("rvalue_str", 42); // 调用移动构造
    std::string name = "lvalue_str";
    auto p2 = my_make_unique<MyClass>(name, 100);          // 调用拷贝构造

    std::cout << "\n=== emplace 模式 ===" << std::endl;
    std::vector<std::string> v;
    std::string str = "example";
    emplaceBack(v, str);           // 拷贝
    emplaceBack(v, std::move(str)); // 移动
    std::cout << "v[0] = " << v[0] << ", v[1] = " << v[1] << std::endl;

    std::cout << "All forward tests passed!" << std::endl;
    return 0;
}

```

8.7 面试高频问题（至少5道+标准答案）

Q1: std::move 和 std::forward 的区别？

答案：std::move 无条件将参数转为右值引用。std::forward 条件转换：如果原始参数是左值则保持左值，是右值则转为右值。std::move 用于你想要移动一个对象时，std::forward 用于转发参数时保持其值类别。记忆技巧：move 做"减法"（去掉左值性），forward 做"保留"（保留原值类别）。

Q2: 什么是转发引用（万能引用）？如何识别？

答案：形式为 T&& 且 T 是被推导的模板参数（或 auto&&）的引用就是转发引用。关键是 T 必须被推导，如果 T 已经确定（如 std::string&&），则是普通的右值引用，不是转发引用。

Q3: 完美转发中的引用折叠规则是什么？

答案：T& &、T& &&、T&& & 都折叠为 T&；只有 T&& && 折叠为 T&&。这四条规则使得转发引用能正确处理左值（T推导为 X&，引用折叠得 X&）和右值（T推导为 X，得 X&&）。

Q4: 为什么 std::forward 的模板参数必须显式指定？

答案：因为 std::forward<T> 的类型转换逻辑依赖于 T：如果 T 是左值引用类型（如 int&），通过引用折叠返回左值引用；如果 T 是非引用类型（如 int），返回右值引用。这个区分必须通过显式指定 T 来实现。

Q5: 为什么 emplace_back 用完美转发而不是重载？

答案：如果使用重载，需要为每种参数组合写多个版本（const T&、T&&、const T& + const U&、T&& + U&&...），组合爆炸。完美转发用一个模板处理所有情况，且参数数量和类型都灵活。

8.8 延伸问题

- 完美转发的开销（编译期和运行期）如何？
- 如何检测完美转发是否正确工作？
- forward 失败的情况（forwarding failure）有哪些？
- C++20 的 `std::forward_like` 是什么场景使用的？

9. 左值与右值 (lvalue/rvalue)

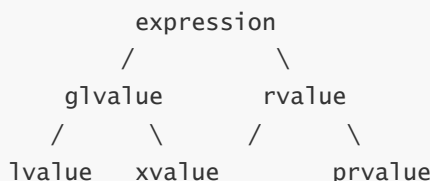
9.1 是什么（定义）

C++ 中每个表达式有两个属性：**类型 (type)** 和**值类别 (value category)**。C++11 将值类别细分为三种：

- **左值 (lvalue)**：有标识（有名字/地址）且不能移动的表达式。可以取地址。
- **将亡值 (xvalue)**：有标识但可以移动的表达式（如 `std::move(x)` 的返回值）。
- **纯右值 (prvalue)**：没有标识的临时对象/字面量/运算结果。

泛左值 (glvalue) = lvalue + xvalue（有标识的表达式）

右值 (rvalue) = prvalue + xvalue（可以移动的表达式）



记忆口诀：

lvalue -> 有地址，不能移动
xvalue -> 有地址，即将被移动
prvalue -> 无地址，纯临时
rvalue -> xvalue + prvalue（都可以绑定到 T&&）

9.2 为什么会出现

1. **移动语义需要区分可移动对象**：需要知道哪些对象可以安全地"窃取"资源。
2. **完美转发需要保留值类别**：函数模板需要知道参数原始是左值还是右值。
3. **C++11 之前的分类不够用**：C++98 只有左值和右值，无法区分 T&& 引用的临时对象和将被移动的具名对象。

9.3 底层原理

判断方法：

能取地址 (&expr) -> lvalue 或 xvalue
+-- 不是 T&& 类型或不是即将销毁 -> lvalue
+-- T&& 类型且即将销毁 -> xvalue
不能取地址 -> prvalue

常见值类别示例：

```

int x = 10;
x;           // lvalue
&x;         // 可以取地址

42;          // prvalue (字面量)
x + 5;       // prvalue (临时结果)
std::string(""); // prvalue (临时对象)

std::move(x); // xvalue (将亡值)
static_cast<int&&>(x); // xvalue

int&& r = 42;
r;           // lvalue! (有名字的右值引用是左值)

```

9.4 优缺点比较表

值类别	可寻址	可移动	示例
lvalue	是	否	变量名、解引用指针、 <code>*this</code>
xvalue	是	是	<code>std::move(x)</code> 、 <code>static_cast<T&&>(x)</code>
prvalue	否	是	字面量、临时对象、函数返回非引用

9.5 典型应用

```

// 1. 移动语义 (xvalue 参与)
std::string s1 = std::move(s2); // s2 -> xvalue, 触发移动构造

// 2. 转发引用 (绑定任何值类别)
auto&& r1 = 42; // prvalue -> r1 是 int&&
auto&& r2 = x; // lvalue -> r2 是 int&
auto&& r3 = std::move(x); // xvalue -> r3 是 int&&

// 3. 重载决议
void f(const T&); // 接受所有值类别 (const 引用可以绑定到右值)
void f(T&&); // 只接受 rvalue (优先匹配)

```

9.6 完整代码示例

```

#include <iostream>
#include <string>
#include <utility>
#include <type_traits>

// 重载: 区分左值和右值
void overload(int&) { std::cout << " called lvalue overload" << std::endl; }
void overload(int&&) { std::cout << " called rvalue overload" << std::endl; }

int getInt() { return 42; }
int& getRef(int& x) { return x; }

```



```

int&& getRvalRef() { return 42; }

int main() {
    std::cout << "=== 基本值类别 ===" << std::endl;
    int x = 10;
    int& rx = x;
    int&& rval = 42;

    overload(x);           // lvalue
    overload(rx);          // lvalue (引用变量本身是左值)
    overload(rval);        // lvalue! (有名字的右值引用是左值)
    overload(42);          // rvalue (字面量)
    overload(std::move(x)); // rvalue (xvalue)
    overload(x + 1);       // rvalue (prvalue)

    std::cout << "\n=== 引用绑定 ===" << std::endl;
    int& lr = x;           // 左值引用绑定左值
    const int& clr = 42;    // const左值引用可以绑定右值
    int&& rr = 42;          // 右值引用绑定右值
    int&& rr2 = std::move(x); // 右值引用绑定xvalue
    // int&& rr3 = x;        // 错误: 右值引用不能绑定左值

    std::cout << "\n=== 转发引用 ===" << std::endl;
    auto&& ar1 = x;         // 绑定左值 -> int&
    auto&& ar2 = 42;        // 绑定右值 -> int&&
    static_assert(std::is_lvalue_reference_v<decltype(ar1)>);
    static_assert(std::is_rvalue_reference_v<decltype(ar2)>);

    std::cout << "\n=== std::move 返回 xvalue ===" << std::endl;
    int* px = &x;          // &x 可以取地址, x是左值
    // int* py = &std::move(x); // 错误: xvalue 不能取地址!
    // 但 xvalue 类型上是右值引用, 可以移动

    std::cout << "\n=== 关键规则记忆 ===" << std::endl;
    std::cout << "1. 有名字的右值引用变量本身是左值" << std::endl;
    std::cout << "2. std::move 返回 xvalue (有标识但可移动)" << std::endl;
    std::cout << "3. 临时对象和字面量是 prvalue" << std::endl;
    std::cout << "4. 左值引用(T&)只绑定左值" << std::endl;
    std::cout << "5. 右值引用(T&&)只绑定右值" << std::endl;
    std::cout << "6. const左值引用(const T&)可以绑定任何东西" << std::endl;

    std::cout << "All value category tests passed!" << std::endl;
    return 0;
}

```

9.7 面试高频问题 (至少5道+标准答案)

Q1: 什么是左值和右值? 最简判断方法?

答案: 左值是可以取地址的表达式(持久对象), 右值是不能取地址的临时对象或字面量。最简判断: 能用 `&expr` 的是左值或 xvalue; 不能的是 prvalue。在 C++11 中, 右值 = prvalue + xvalue。

Q2: 为什么"有名字的右值引用是左值"?

答案: `int&& r = 42;` 中 `r` 是一个变量名, 有标识、有地址、持久存在, 所以它是左值。右值引用只表示它绑定到一个右值, 但变量本身是左值。这是最经典的陷阱: 如果想把 `r` 当作右值传递, 需要 `std::move(r)`。

Q3: `const T&` 和 `T&&` 在绑定规则上有什么区别?

答案: `const T&` 可以绑定到所有值类别 (lvalue、xvalue、prvalue), 因此常用于只读参数。`T&&` 只能绑定到 rvalue (xvalue + prvalue), 用于移动语义或重载。

Q4: 如何判断一个表达式是 xvalue?

答案: 同时满足两个条件: 有标识 (是 glvalue) 但资源可以复用。具体地: (1) 返回类型是 `T&&` 的函数调用 (如 `std::move`); (2) 转换为右值引用的 `cast` 表达式; (3) 访问 xvalue 对象的成员 (成员也是 xvalue)。

Q5: `auto&&` 和 `T&&` 有什么区别?

答案: `auto&&` 是变量声明的转发引用, `T&&` (在模板中 `T` 被推导) 是模板参数的转发引用。两者行为相同: 可以绑定任何值类别, 左值推导出 `T&`, 右值推导出 `T&&`。`auto&&` 常用于泛型 lambda 和范围 for 循环。

9.8 延伸问题

- 值类别和 move 语义如何影响函数重载决议?
- prvalue 与 C++17 的"临时对象实化 (temporary materialization)"有什么关系?
- 为什么 range-for 中推荐 `auto&&`?
- C++17 guaranteed copy elision 与值类别的关系?

10. RAII 资源获取即初始化

10.1 是什么 (定义)

RAII (Resource Acquisition Is Initialization) 是 C++ 中最重要的资源管理范式。核心思想: 在构造函数中获取资源, 在析构函数中释放资源, 利用 C++ 确定性的对象生命周期来保证资源不泄漏。

```
class FileHandle {
    FILE* fp_;
public:
    FileHandle(const char* path) : fp_(fopen(path, "r")) {} // 获取资源
    ~FileHandle() { if (fp_) fclose(fp_); }                // 释放资源
    // 禁止拷贝, 允许移动...
};
```

10.2 为什么会出现

1. **异常安全**: C++ 中异常可能导致跳过资源释放代码, RAII 利用栈展开 (stack unwinding) 自动调用析构函数保证释放。
2. **避免资源泄漏**: 手动管理 `new/delete`、`fopen/fclose`、`lock/unlock` 极易出错。
3. **简化代码**: 不需要在每个退出路径写释放代码 (包括 `return`、异常、`goto`)。
4. **确定性析构**: C++ 对象的生命周期完全由作用域决定, 离开作用域即刻析构。

10.3 底层原理

RAII 的工作机制：

1. 栈展开（Stack Unwinding）：

当异常抛出时，运行时系统从 `throw` 点到 `catch` 点之间的所有局部对象会被自动析构（按构造的逆序）。

2. 确定性析构：

对象离开作用域（`}`）时，析构函数被确定性地调用。

这是 C++ 区别于 GC 语言的核心优势之一。

内存布局示意：

```
{
    std::lock_guard<std::mutex> lock(mtx); // 构造：获取锁
    // ... 可能抛异常的代码 ...
    if (error) return; // 提前返回 → lock析构 → 释放锁
    // ... 更多代码 ...
} // 离开作用域 → lock析构 → 自动释放锁
```

经典RAII包装：

资源类型	RAII包装器
堆内存	<code>std::unique_ptr</code> / <code>std::shared_ptr</code>
文件句柄	<code>std::fstream</code> / <code>std::ifstream</code> / <code>std::ofstream</code>
互斥锁	<code>std::lock_guard</code> / <code>std::unique_lock</code> / <code>std::scoped_lock</code>
数据库连接	自定义 <code>ConnectionGuard</code>
Socket	自定义 <code>SocketGuard</code>
GDI对象	自定义 <code>GdiGuard</code>

10.4 优缺点

优点	缺点
异常安全（核心优势）	需要为每类资源编写 RAII 包装类
确定性释放，无 GC 停顿	不适合生命周期跨越多个作用域的资源（需智能指针配合）
代码简洁，消除重复释放代码	循环引用场景需要 <code>weak_ptr</code> 配合
零运行时开销（编译期确定）	对新手有一定学习曲线
支持 move-only 资源（ <code>unique_ptr</code> ）	析构函数不能抛异常（会引发 <code>std::terminate</code> ）

10.5 典型应用

```
// 1. 智能指针
{
    auto ptr = std::make_unique<Resource>();
    ptr->dowork();
    // throw可能发生
} // ptr自动delete
```

```

// 2. 锁管理
std::mutex mtx;
{
    std::lock_guard<std::mutex> lock(mtx);
    // 临界区
} // 自动unlock

// 3. 文件流
{
    std::ifstream file("data.txt");
    std::string line;
    while (std::getline(file, line)) { /* ... */ }
} // file自动关闭

// 4. 事务管理
class Transaction {
    Database& db_;
public:
    Transaction(Database& db) : db_(db) { db_.begin(); }
    ~Transaction() { if (!committed_) db_.rollback(); }
    void commit() { db_.commit(); committed_ = true; }
private:
    bool committed_ = false;
};

```

10.6 完整代码示例

```

#include <iostream>
#include <fstream>
#include <mutex>
#include <memory>
#include <stdexcept>

// === 自定义 RAII 文件句柄 ===
class FileGuard {
    FILE* fp_;
public:
    explicit FileGuard(const char* path, const char* mode)
        : fp_(fopen(path, mode)) {
        if (!fp_) throw std::runtime_error("Failed to open file");
        std::cout << "File opened" << std::endl;
    }

    ~FileGuard() {
        if (fp_) {
            fclose(fp_);
            std::cout << "File closed" << std::endl;
        }
    }
}

// 禁止拷贝，允许移动

```

```

FileGuard(const FileGuard&) = delete;
FileGuard& operator=(const FileGuard&) = delete;
FileGuard(FileGuard&& other) noexcept : fp_(other.fp_) {
    other.fp_ = nullptr;
}

void write(const std::string& data) {
    if (fp_) fputs(data.c_str(), fp_);
}
};

// === 自定义 RAII 数据库事务 ===
class TransactionGuard {
    bool active_;
    bool committed_;
public:
    TransactionGuard() : active_(true), committed_(false) {
        std::cout << "Transaction BEGIN" << std::endl;
    }

    ~TransactionGuard() {
        if (active_ && !committed_) {
            std::cout << "Transaction ROLLBACK (auto)" << std::endl;
        }
    }

    void commit() {
        if (active_) {
            committed_ = true;
            std::cout << "Transaction COMMIT" << std::endl;
        }
    }

    void rollback() {
        if (active_) {
            std::cout << "Transaction ROLLBACK (manual)" << std::endl;
            active_ = false;
        }
    }
};

// === RAII 锁 ===
std::mutex g_mtx;

void raiiLockDemo() {
    std::lock_guard<std::mutex> lock(g_mtx);
    std::cout << " Critical section (lock held)" << std::endl;
    // 离开作用域自动释放锁
}

// === 演示异常安全 ===
void mightThrow() {
    auto ptr = std::make_unique<int>(42); // RAII 内存

```

```

TransactionGuard txn;                                // RAII 事务

std::cout << "  Working..." << std::endl;
throw std::runtime_error("Something went wrong");
// ptr 和 txn 都会自动清理！（栈展开）

txn.commit(); // 永远不会执行到
}

int main() {
    std::cout << "=== 文件 RAII ===" << std::endl;
    {
        FileGuard fg("test_output.txt", "w");
        fg.write("Hello, RAII!\n");
    } // 自动关闭文件

    std::cout << "\n=== 锁 RAII ===" << std::endl;
    raiiLockDemo();
    std::cout << "  Lock released" << std::endl;

    std::cout << "\n=== 异常安全演示 ===" << std::endl;
    try {
        mightThrow();
    } catch (const std::exception& e) {
        std::cout << "  Caught: " << e.what() << std::endl;
        std::cout << "  (All RAII resources were automatically cleaned up!)" << std::endl;
    }

    std::cout << "\n=== 事务显式提交 ===" << std::endl;
    {
        TransactionGuard txn;
        txn.commit(); // 显式提交，不会回滚
    }

    std::cout << "\nAll RAII tests passed!" << std::endl;
    return 0;
}

```

10.7 面试高频问题（至少5道+标准答案）

Q1: 什么是 RAII? 核心思想是什么?

答案：RAII (Resource Acquisition Is Initialization) 是 C++ 资源管理的核心范式。核心思想：将资源的生命周期绑定到对象的生命周期——构造函数获取资源，析构函数释放资源。利用 C++ 确定性的对象析构机制保证资源在任何情况下都不会泄漏（包括异常）。

Q2: RAII 如何保证异常安全?

答案：当异常抛出时，C++ 进行栈展开 (stack unwinding)，从 throw 点到 catch 点之间的所有局部对象按构造的逆序自动调用析构函数。如果资源由 RAII 对象管理，析构函数会释放资源，无需手动 cleanup。这是“不会泄漏”的保证。

Q3: RAII 和 Java/C# 的 try-finally / using 有什么区别?

答案：RAII 是语言级别内置的（编译器自动插入析构调用），不需要开发者显式写 finally 块。C# 的 `using` 语句也需要显式声明。RAII 更彻底：无处不在、默认安全；GC 语言的资源管理需要开发者主动配合。

Q4: 析构函数中能抛异常吗？为什么？

答案：绝对不能！C++ 标准规定：如果栈展开过程中（正在处理一个异常）析构函数又抛出异常，`std::terminate` 会被调用，程序直接终止。即使没有在处理异常，析构函数抛异常也会导致资源泄漏（后续的对象不会被析构）。析构函数应标记为 `noexcept`。

Q5: 列举 C++ 标准库中的 RAII 类。

答案：(1) 内存：`std::unique_ptr`、`std::shared_ptr`、`std::weak_ptr`；(2) 锁：`std::lock_guard`、`std::unique_lock`、`std::scoped_lock(C++17)`；(3) 文件：`std::ifstream`、`std::ofstream`、`std::fstream`；(4) 线程：`std::jthread(C++20)`（自动 join）。

Q6: 如何为 C 风格 API 设计 RAII 包装？

答案：经典模式：

```
class CResourceGuard {
    HANDLE h_;
public:
    CResourceGuard(args) : h_(CreateResource(args)) {}
    ~CResourceGuard() { if(h_ != INVALID_HANDLE) CloseResource(h_); }
    CResourceGuard(const CResourceGuard&) = delete;
    CResourceGuard& operator=(const CResourceGuard&) = delete;
    CResourceGuard(CResourceGuard&& o) noexcept : h_(o.h_) { o.h_ = INVALID_HANDLE; }
};
```

10.8 延伸问题

- RAII 和 scope guard 模式（如 `folly::ScopeGuard`）的关系？
- 如何用 RAII 实现引用计数？
- C++17 的 `std::scoped_lock` 相比 `std::lock_guard` 有什么改进？
- 在构造函数失败后，已经构造好的成员如何析构？

11. 智能指针 (Smart Pointers)

11.1 是什么（定义）

智能指针是 RAII 范式在指针管理上的应用，封装了原始指针并提供自动内存管理。C++11 引入三种标准智能指针：

```
#include <memory>

// unique_ptr: 独占所有权, 不能拷贝, 只能移动
std::unique_ptr<int> up = std::make_unique<int>(42);

// shared_ptr: 共享所有权, 引用计数
std::shared_ptr<int> sp = std::make_shared<int>(42);

// weak_ptr: 弱引用, 不增加引用计数, 用于打破循环引用
std::weak_ptr<int> wp = sp;
```

特性	unique_ptr	shared_ptr	weak_ptr
所有权	独占	共享	无（观察者）
拷贝	禁止	允许（引用计数+1）	允许
移动	允许（转移所有权）	允许	允许
内存开销	0（默认删除器）	16字节（指针+控制块）	同shared_ptr
线程安全	对象本身不共享	控制块线程安全	-
典型场景	工厂函数、PIMPL	共享资源、图结构	打破循环引用

11.2 为什么会出现

- 1. **消除手动 delete**: new/delete 配对极易遗漏，尤其是在异常或多 return 路径下。
- 2. **明确所有权语义**: 代码自文档化——看类型就知道所有权模型（独占/共享/观察）。
- 3. **避免悬空指针和双重释放**: 自动析构保证只释放一次。
- 4. **异常安全**: 智能指针是 RAII 对象，栈展开时自动释放。

11.3 底层原理

```
unique_ptr 内部结构（简化）：
template<typename T, typename Deleter = std::default_delete<T>>
class unique_ptr {
    T* ptr_;          // 原始指针（8字节）
    Deleter d_;       // 删除器（若为空基类则0字节，EBO优化）
public:
    ~unique_ptr() { if(ptr_) d_(ptr_); }
    T* release() noexcept { T* p = ptr_; ptr_ = nullptr; return p; }
    // 禁止拷贝, 允许移动
};

shared_ptr 内部结构:
shared_ptr<T> 对象（16字节）:
    --- T* ptr_          （8字节: 指向被管理对象）
    --- control_block*   （8字节: 指向控制块）

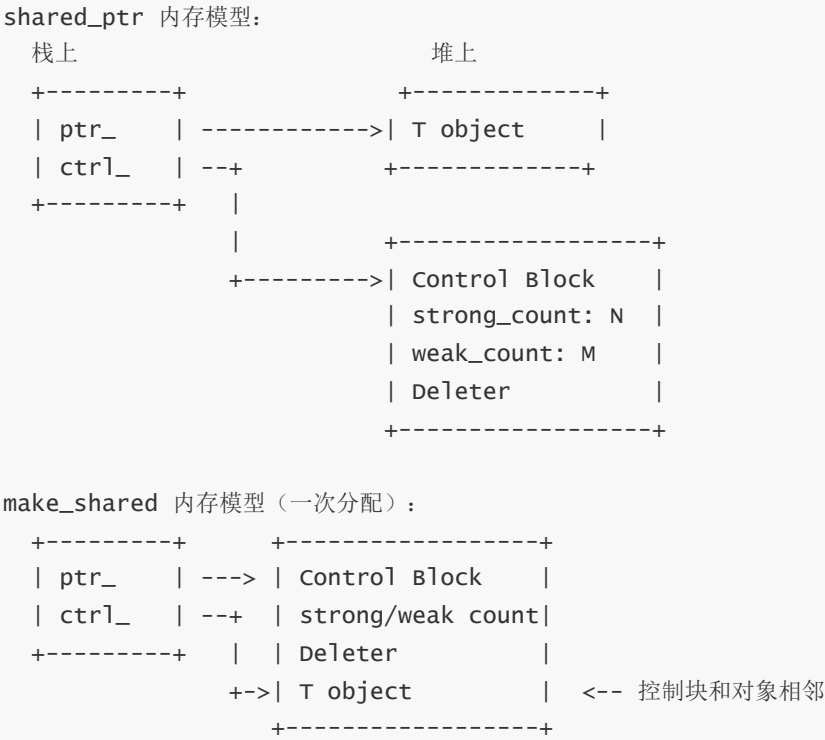
控制块（堆上分配）:
```


+++ strong_count (引用计数)
+++ weak_count (弱引用计数)
+++ Deleter (删除器，类型擦除)
+++ Allocator (分配器)

make_shared 的优势：一次分配对象+控制块（节省一次堆分配）

weak_ptr ：
+++ 不增加 strong_count
+++ 增加 weak_count（控制块在weak_count>0时保持存活）
+++ lock() 返回 shared_ptr（如果对象还在）或空（如果已释放）

关键内存布局（ASCII图）：



优点	缺点
共享所有权，自动化引用计数	16字节额外开销（指针+控制块）
支持自定义删除器（类型擦除）	循环引用导致内存泄漏（需 weak_ptr）
控制块线程安全	多线程修改指向对象需外部同步
可从 this 安全创建（enable_shared_from_this）	make_shared 的弱引用会延迟对象释放

weak_ptr:

优点	缺点
打破循环引用	不能直接解引用（需 lock()）
安全检测对象是否存活	额外的控制块开销
可用于缓存、观察者模式	-

11.5 典型应用

```
// 1. unique_ptr: 工厂函数
std::unique_ptr<Widget> createWidget() {
    return std::make_unique<Widget>();
}

// 2. unique_ptr: PIMPL 模式
class MyClass {
    struct Impl;
    std::unique_ptr<Impl> pImpl_;
public:
    MyClass();
    ~MyClass(); // 必须在 .cpp 中定义（Impl 不完整类型）
};

// 3. shared_ptr: 共享资源
class Texture {
    std::shared_ptr<TextureData> data_;
public:
    Texture(const Texture&) = default; // 浅拷贝，共享底层数据
};

// 4. weak_ptr: 打破循环引用
class Node {
    std::vector<std::shared_ptr<Node>> children_;
    std::weak_ptr<Node> parent_; // 弱引用父节点
};

// 5. weak_ptr: 缓存
std::unordered_map<int, std::weak_ptr<Resource>> cache;
auto sp = cache[key].lock();
```

```

if (sp) { return sp; } // 资源还在
else { /* 重新加载 */ }

// 6. unique_ptr + 自定义删除器
auto deleter = [](FILE* f) { if(f) fclose(f); };
std::unique_ptr<FILE, decltype(deleter)> file(fopen("a.txt", "r"), deleter);

```

11.6 完整代码示例

```

#include <iostream>
#include <memory>
#include <vector>
#include <cassert>

struct Resource {
    int id;
    Resource(int i) : id(i) { std::cout << " Resource(" << id << ") created" << std::endl; }
}

~Resource() { std::cout << " Resource(" << id << ") destroyed" << std::endl; }
void work() { std::cout << " Resource(" << id << ") working" << std::endl; }
};

// === unique_ptr 演示 ===
void uniquePtrDemo() {
    std::cout << "=== unique_ptr ===" << std::endl;
    auto up1 = std::make_unique<Resource>(1);

    // 移动所有权
    auto up2 = std::move(up1);
    assert(up1 == nullptr); // up1 已空
    up2->work();

    // 容器中的 unique_ptr
    std::vector<std::unique_ptr<Resource>> vec;
    vec.push_back(std::make_unique<Resource>(2));
    vec.push_back(std::make_unique<Resource>(3));
    // 手动释放 (return之前自动释放)
    for (auto& up : vec) up->work();
}

// === shared_ptr 演示 ===
void sharedPtrDemo() {
    std::cout << "\n=== shared_ptr ===" << std::endl;
    auto sp1 = std::make_shared<Resource>(10);
    std::cout << " use_count: " << sp1.use_count() << std::endl; // 1

    {
        auto sp2 = sp1; // 共享所有权
        std::cout << " use_count: " << sp1.use_count() << std::endl; // 2
        sp2->work();
    } // sp2 析构
}

```

```

std::cout << " use_count: " << sp1.use_count() << std::endl; // 1

// reset
sp1.reset(); // 手动释放
std::cout << " After reset, use_count (from expired): "
    << sp1.use_count() << std::endl; // 0
}

// === 循环引用问题 ===
struct CycleNode {
    std::string name;
    std::shared_ptr<CycleNode> friend_;
    std::weak_ptr<CycleNode> weak_friend_; // 弱引用

    CycleNode(const std::string& n) : name(n) {
        std::cout << " CycleNode(" << name << ") created" << std::endl;
    }
    ~CycleNode() {
        std::cout << " CycleNode(" << name << ") destroyed" << std::endl;
    }
};

void cycleRefDemo() {
    std::cout << "\n=== 循环引用 ===" << std::endl;

    {
        std::cout << " Bad (shared_ptr循环引用, 不会释放):" << std::endl;
        auto a = std::make_shared<CycleNode>("A");
        auto b = std::make_shared<CycleNode>("B");
        a->friend_ = b;
        b->friend_ = a; // 循环引用! 离开作用域后 A 和 B 都不会析构
    } // 内存泄漏!

    {
        std::cout << " Good (weak_ptr打破循环引用):" << std::endl;
        auto a = std::make_shared<CycleNode>("C");
        auto b = std::make_shared<CycleNode>("D");
        a->friend_ = b; // shared_ptr, 正常引用
        b->weak_friend_ = a; // weak_ptr, 不增加引用计数
        // 离开作用域后 C 和 D 正常析构
    }

    std::cout << " End of cycleRefDemo scope" << std::endl;
}

// === enable_shared_from_this ===
class SelfAware : public std::enable_shared_from_this<SelfAware> {
public:
    std::shared_ptr<SelfAware> getShared() {
        return shared_from_this(); // 安全地从 this 创建 shared_ptr
    }
};

```

```

void enableSharedDemo() {
    std::cout << "\n=== enable_shared_from_this ===" << std::endl;
    auto sp = std::make_shared<SelfAware>();
    auto sp2 = sp->getShared();
    std::cout << "    use_count: " << sp.use_count() << std::endl; // 2
}

// === weak_ptr 缓存模式 ===
void weakPtrCache() {
    std::cout << "\n=== weak_ptr 缓存 ===" << std::endl;
    std::weak_ptr<Resource> cache;

    {
        auto res = std::make_shared<Resource>(99);
        cache = res;

        if (auto locked = cache.lock()) {
            std::cout << "    Cache hit! Resource still alive." << std::endl;
            locked->work();
        }
    } // res 析构

    if (auto locked = cache.lock()) {
        std::cout << "    Cache hit!" << std::endl;
    } else {
        std::cout << "    Cache miss! Resource already destroyed." << std::endl;
    }
}

int main() {
    uniquePtrDemo();
    sharedPtrDemo();
    cycleRefDemo();
    enableSharedDemo();
    weakPtrCache();

    std::cout << "\nAll smart pointer tests completed!" << std::endl;
    return 0;
}

```

11.7 面试高频问题（至少5道+标准答案）

Q1: unique_ptr 和 shared_ptr 的区别？各自使用场景？

答案：unique_ptr 独占所有权，不能拷贝只能移动，内存开销为 0（默认删除器）或删除器大小。适合：工厂函数返回值、PIMPL 模式、容器中管理多态对象、明确独占所有权的场景。shared_ptr 共享所有权，使用引用计数，大小为 16 字节（指针+控制块指针）。适合：多个对象共享同一资源、图/树结构中的节点、异步回调中需保持对象存活。

Q2: shared_ptr 的引用计数如何实现线程安全？

答案：控制块中的 `strong_count` 和 `weak_count` 使用原子操作（`std::atomic`）实现增减，保证多线程下引用计数的正确性。但对象本身的数据读写不受保护，需要外部同步。注意：不是所有操作都是线程安全的——多个线程同时读同一个 `shared_ptr` 可能需要额外同步。

Q3: `make_shared` 和 `new shared_ptr` 的区别？

答案：(1) `make_shared` 只做一次堆分配（对象和控制块一起分配），`new shared_ptr<T>(new T)` 做两次分配。(2) `make_shared` 异常安全——如果对象构造过程中抛异常，不会泄漏。(3) `make_shared` 的缺点：只要有 `weak_ptr` 存在，整个内存块（包括对象）都不会释放（即使 `strong_count=0`），因为控制块和对象在同一块内存中。

Q4: `weak_ptr` 的作用和使用场景？

答案：`weak_ptr` 是对 `shared_ptr` 管理对象的弱引用，不增加引用计数，用于：(1) 打破循环引用（如树节点的 `parent` 指针、双向链表）；(2) 实现缓存（资源不用时可被释放，使用时通过 `lock()` 检查是否还存活）；(3) 观察者模式（观察者不需要保持被观察者存活）。

Q5: `enable_shared_from_this` 是什么？为什么要用它？

答案：当需要从 `this` 安全地创建 `shared_ptr` 时使用。如果直接用 `std::shared_ptr<T>(this)`，会创建多个独立控制块导致双重释放。`enable_shared_from_this` 内部存储一个 `weak_ptr`，`shared_from_this()` 通过它返回共享同一控制块的 `shared_ptr`。前提：调用时对象必须已被 `shared_ptr` 管理。

Q6: `shared_ptr` 能否管理数组？`unique_ptr` 呢？

答案：`shared_ptr<T[]>` 在 C++17 中支持数组，可以正确调用 `delete[]`。`unique_ptr<T[]>` 的偏特化版本也支持数组。但推荐用 `std::vector` 或 `std::array` 替代动态数组。

Q7: `unique_ptr` 和 `shared_ptr` 的删除器有什么不同？

答案：`unique_ptr` 的删除器是模板参数的一部分，存储在对象内部（EBO优化）。`shared_ptr` 的删除器通过类型擦除存储，不占据 `shared_ptr` 对象本身的空间（存储在控制块中）。因此，具有不同删除器的 `shared_ptr<T>` 可以是同一类型，而具有不同删除器的 `unique_ptr<T>` 是不同类型。

11.8 延伸问题

- 如何在线程间安全地传递 `shared_ptr`？
 - `shared_ptr` 的 `aliasing constructor` 是什么用途？
 - C++20 的 `std::atomic<std::shared_ptr<T>>` 解决了什么问题？
 - `unique_ptr` 如何与 C 风格 API 交互（`get()`、`release()`）？
 - `std::make_unique` 为何到 C++14 才引入？
-

12. vector 动态数组

12.1 是什么（定义）

`std::vector` 是 C++ 标准库中最常用的容器，封装了**动态数组**。元素在内存中**连续存储**，支持 $O(1)$ 随机访问，在尾部插入/删除均摊 $O(1)$ ，在其他位置插入/删除 $O(n)$ 。

```
// 定义在 <vector>
std::vector<int> v;           // 空容器
std::vector<int> v2(5, 10);   // 5个元素，每个值为10
std::vector<int> v3{1, 2, 3, 4}; // 初始化列表
```

12.2 为什么会出现

1. 取代 C 风格数组：自动管理内存，支持动态扩容。
2. 需要连续内存的场景：缓存友好的数据布局、与 C API 交互。
3. 最高效的顺序容器：大多数场景下 `vector` 是最佳默认选择。

12.3 底层原理

`vector` 的内存布局：

```
+-----+-----+-----+-----+
| elem[0] | elem[1] | elem[2] | ... | elem[n] | <-- 连续内存
+-----+-----+-----+-----+
^               ^               ^
begin()         end()           capacity()
```

内部三个指针（典型实现）：

```
T* start_;    // 指向第一个元素
T* finish_;   // 指向最后一个元素的下一个位置
T* end_of_storage_; // 指向已分配内存的末尾
```

扩容机制（gcc/clang，增长因子为2）：

1. 分配新内存：`new_capacity = old_capacity * 2`
2. 移动/拷贝旧元素到新内存
3. 释放旧内存
4. 更新三个指针

时间复杂度：

随机访问： $O(1)$
尾部插入：均摊 $O(1)$
其他位置插入/删除： $O(n)$
查找： $O(n)$ （未排序时）

扩容因子为什么是 2（或 1.5）？

gcc 用 2，msvc 用 1.5。因子为 2 可能导致“永远无法复用已释放内存”（每次扩容的新大小总是大于之前所有释放的总和）。因子 1.5 可以复用（ $n * 1.5^k$ 的序列最终可以容纳之前的块）。但实际上这不是大问题，因为 `malloc` 有自己的分配策略。

12.4 优缺点

优点	缺点
连续内存，缓存友好	中间插入/删除 $O(n)$
$O(1)$ 随机访问	扩容时所有指针/引用/迭代器失效
尾部插入均摊 $O(1)$	<code>push_front</code> 不存在（需 <code>insert(begin)</code> ， $O(n)$ ）
与 C API 兼容（ <code>v.data()</code> ）	扩容时的内存重分配开销
内存紧凑（相比 <code>list</code> ）	<code>bool</code> 特化有坑（ <code>vector</code> 是位压缩）

12.5 典型应用

```
// 1. 动态数组（默认容器首选）
std::vector<int> scores;
scores.reserve(1000); // 预分配，避免多次扩容

// 2. 与 C API 交互
c_api_function(v.data(), v.size());

// 3. 栈/队列实现
std::vector<int> stack;
stack.push_back(1); stack.pop_back();

// 4. 批量处理
v.insert(v.end(), other.begin(), other.end());
v.erase(std::remove_if(v.begin(), v.end(), pred), v.end());
```

12.6 完整代码示例

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <chrono>

int main() {
    // === 构造和基本操作 ===
    std::vector<int> v = {1, 2, 3, 4, 5};

    v.push_back(6); // 尾部插入
    v.pop_back();   // 尾部删除
    v.insert(v.begin(), 0); // 头部插入  $O(n)$ 
    v.erase(v.begin() + 2); // 删除第3个元素

    std::cout << "v: ";
    for (int n : v) std::cout << n << " ";
    std::cout << std::endl;
```



```

// === 容量管理 ===
std::cout << "size: " << v.size()
          << ", capacity: " << v.capacity() << std::endl;

v.shrink_to_fit();    // 请求释放多余内存（不保证）
std::cout << "After shrink_to_fit, capacity: "
          << v.capacity() << std::endl;

// === reserve 预分配 ===
std::vector<int> vr;
vr.reserve(10000);    // 预分配，避免多次扩容
std::cout << "After reserve, capacity: " << vr.capacity() << std::endl;

// === 数据指针 ===
int* raw = v.data();
std::cout << "First element via data(): " << raw[0] << std::endl;

// === emplace_back ===
std::vector<std::pair<int, std::string>> pairs;
pairs.emplace_back(1, "hello"); // 原地构造，无临时对象
std::cout << "emplace_back: {" << pairs[0].first
          << ", " << pairs[0].second << "}" << std::endl;

// === erase-remove 惯用法 ===
std::vector<int> nums = {1, 2, 3, 2, 4, 2, 5};
nums.erase(std::remove(nums.begin(), nums.end(), 2), nums.end());
std::cout << "After remove 2: ";
for (int n : nums) std::cout << n << " ";
std::cout << std::endl;

// === clear + shrink ===
v.clear();
std::cout << "After clear: size=" << v.size()
          << ", capacity=" << v.capacity() << std::endl;
// 释放内存的技巧
std::vector<int>().swap(v); // 与空vector交换，彻底释放内存

std::cout << "All vector tests passed!" << std::endl;
return 0;
}

```

12.7 面试高频问题

Q1: vector 的扩容机制是怎样的？扩容因子是多少？

答案：当 `size() == capacity()` 时触发扩容，新容量通常是旧容量的 2 倍（gcc）或 1.5 倍（msvc）。过程：分配新内存 -> 移动/拷贝旧元素 -> 释放旧内存。扩容导致所有迭代器、指针、引用失效。可通过 `reserve()` 预分配避免不必要的扩容。

Q2: vector 的 `push_back` 和 `emplace_back` 有什么区别？

答案：`push_back` 接收已构造的对象（可能发生拷贝或移动），`emplace_back` 直接在容器内原地构造对象（用完完美转发传递参数给构造函数）。`emplace_back` 消除了临时对象的创建和移动/拷贝开销。

Q3: vector 有什么特殊之处（陷阱）？

答案：`vector<bool>` 是标准要求的特化版本，将 `bool` 压缩为位存储，每个元素占 1 bit。后果：(1) 不满足标准容器要求（不能返回 `bool&`，返回代理对象）；(2) `v[0]` 返回代理而非 `bool&`，`auto b = v[0]` 的类型推断可能出乎意料；(3) 性能反而更差。建议用 `vector<char>` 或 `deque<bool>` 替代。

Q4: 如何正确释放 vector 占用的内存？

答案：`clear()` 只清空元素（调用析构），不释放内存。释放内存的方法：(1) `v.shrink_to_fit()`（只是请求，不保证）；(2) `std::vector<T>().swap(v)`（与临时空 `vector` 交换，旧内存被临时对象析构时释放）；(3) `v = {}` 然后 `v.shrink_to_fit()`。

Q5: vector 扩容时使用移动还是拷贝？

答案：如果元素的移动构造函数是 `noexcept`，`vector` 扩容时使用移动；否则使用拷贝（保证异常安全）。

12.8 延伸问题

- `vector` 与 `deque` 如何选择？
- 多线程下 `vector` 的并发读写安全问题？
- C++20 的 `std::span` 与 `vector` 的关系？

13. deque 双端队列

13.1 是什么（定义）

`std::deque`（double-ended queue）支持在**头部和尾部**进行 $O(1)$ 插入/删除，元素**不是连续存储**而是分段连续（chunked），支持 $O(1)$ 随机访问（但比 `vector` 稍慢）。

```
std::deque<int> dq = {1, 2, 3};
dq.push_front(0);    // O(1)
dq.push_back(4);     // O(1)
```

13.2 为什么会出现

需要同时支持高效头部和尾部操作的场景，且仍然需要随机访问。`vector` 只有 `push_back` 是 $O(1)$ ，`list` 没有随机访问。

13.3 底层原理

`deque` 的分段连续内存模型：

```
+-----+-----+-----+-----+
| Chunk0 | Chunk1 | Chunk2 | Chunk3 | <-- 中控器（指针数组）
+-----+-----+-----+-----+
      |           |           |           |
      v           v           v           v
[elem...] [elem...] [elem...] [elem...] <-- 每块大小固定（通常512字节/元素大小）
```

`push_front`：在第一个 `chunk` 前面插入，若满则在前面新增 `chunk`

`push_back`：在最后一个 `chunk` 后面插入，若满则在后面新增 `chunk`

`operator[]`: 通过中控器定位 `chunk + 偏移量` (比 `vector` 多一次间接寻址)

对比 `vector` 扩容:

`vector`: 扩容时整个连续块重新分配, 所有元素移动

`deque`: 扩容只需新建 `chunk`, 无需移动已有元素 (迭代器不失效)

13.4 优缺点

对比项	<code>vector</code>	<code>deque</code>
内存布局	连续	分段连续
<code>push_front</code>	$O(n)$	$O(1)$
<code>push_back</code>	均摊 $O(1)$	$O(1)$
随机访问	更快 (单次寻址)	稍慢 (两次寻址)
扩容时迭代器失效	全部失效	不失效 (但可能部分操作使所有迭代器失效)
内存占用	紧凑	额外中控器和 <code>chunk</code> 开销
C API 兼容	<code>data()</code> 返回连续内存	不保证连续

13.5 典型应用

```
// 1. 双端可扩展的队列
std::deque<int> dq;
dq.push_front(1); dq.push_back(2);

// 2. 滑动窗口 (两端操作)
while (dq.size() > windowSize) dq.pop_front();

// 3. stack/queue 的默认底层容器
std::stack<int, std::deque<int>> s; // stack默认用deque
std::queue<int> q;                  // queue默认用deque

// 4. 需要随机访问的两端队列
int mid = dq[dq.size() / 2]; // vector做不到push_front, list做不到[]
```

13.6 完整代码示例

```
#include <iostream>
#include <deque>
#include <algorithm>

int main() {
    std::deque<int> dq = {3, 4, 5};

    // O(1) 两端操作
    dq.push_front(2);
    dq.push_front(1);
```

```

dq.push_back(6);
dq.push_back(7);

std::cout << "Deque contents: ";
for (int n : dq) std::cout << n << " ";
std::cout << std::endl;

// 随机访问 O(1)
std::cout << "dq[0]=" << dq[0] << " dq[3]=" << dq[3]
           << " dq[5]=" << dq[5] << std::endl;

// 两端删除
dq.pop_front();
dq.pop_back();
std::cout << "After pop_front/back: ";
for (int n : dq) std::cout << n << " ";
std::cout << std::endl;

// 中间插入 (O(n)，但只移动较少的那一半)
auto mid = dq.begin() + dq.size()/2;
dq.insert(mid, 100);

// 排序
std::sort(dq.begin(), dq.end());

std::cout << "Final deque: ";
for (int n : dq) std::cout << n << " ";
std::cout << std::endl;

std::cout << "All deque tests passed!" << std::endl;
return 0;
}

```

13.7 面试高频问题

Q1: deque 和 vector 的主要区别？何时选 deque？

答案：deque 支持 $O(1)$ 的 `push_front`，扩容时不会使所有迭代器失效，但内存不是完全连续的。选 deque：(1) 需要高效头部插入；(2) 不希望扩容导致迭代器大面积失效；(3) 元素很大，reallocation 代价高。其他情况优先用 vector。

Q2: deque 的内存布局是怎样的？为什么说“分段连续”？

答案：deque 维护一个中控器（map），它是指向多个固定大小 chunk 的指针数组。每个 chunk 存储若干元素。`operator[]` 需要先通过中控器找 chunk，再在 chunk 内定位元素（两次间接寻址）。`push_front/back` 在新 chunk 进行，不需要移动已有数据。

Q3: deque 的随机访问为什么比 vector 慢？

答案：vector 只需一次指针算术运算（`ptr + idx`）。deque 需要：(1) 计算在哪个 chunk；(2) 通过中控器获取 chunk 地址；(3) 在 chunk 内偏移。多了一次内存间接访问，缓存也不如 vector 友好。

14. list 双向链表

14.1 是什么（定义）

`std::list` 是**双向链表**，元素分散存储在独立的节点中，每个节点包含数据和前后指针。在任何位置插入/删除都是 $O(1)$ ，但不支持随机访问。

```
std::list<int> lst = {1, 2, 3, 4, 5};
lst.push_front(0);
lst.push_back(6);
// lst[3]    // 编译错误！list 不支持随机访问
```

14.2 为什么会出现

需要频繁在任意位置插入/删除、且不需要随机访问的场景。相比 `vector/deque`，插入/删除不会使其他迭代器失效。

14.3 底层原理

`list` 的节点结构：

```
+-----+-----+-----+
| prev | data | next |    <-- 双向链表节点
+-----+-----+-----+
```

`list` 维护两个哨兵节点（sentinel node）：

```
+-----+ +-----+ +-----+ +-----+
| head |<--->|elem1|<--->|elem2|<--->| tail |    <-- 循环双向链表
+-----+ +-----+ +-----+ +-----+
```

每个节点独立分配在堆上，通过指针连接。
`size()` 在 C++11 之后是 $O(1)$ （因为要求为 $O(1)$ ）。

内存开销：每个节点额外 2 个指针（`prev/next`），共 16 字节（64 位系统）。

14.4 优缺点

优点	缺点
任意位置插入/删除 $O(1)$	不支持随机访问 $O(n)$
插入/删除不使迭代器失效	每个节点内存开销大（2 个指针）
可高效拼接 <code>splice()</code>	内存不连续，缓存不友好
<code>stable</code> （稳定）	遍历慢（指针跳转不可预测）

何时使用：极频繁的中间插入/删除、需要 `splice` 操作、要求迭代器稳定性。
何时避免：绝大多数情况下 `vector` 更好（即使有中间插入，如果元素少，`vector` 的批量移动也比 `list` 的指针跳转快）。

14.5 典型应用

```
// 1. LRU Cache (利用 splice 移动元素到头部)
lst.splice(lst.begin(), lst, it); // O(1) 移动节点

// 2. 极少用的场景 (大多数时候 vector 更优)
// 需要频繁在序列中间插入且元素很大
```

14.6 完整代码示例

```
#include <iostream>
#include <list>
#include <algorithm>

int main() {
    std::list<int> lst = {2, 3, 4};

    // 两端操作 O(1)
    lst.push_front(1);
    lst.push_back(5);

    std::cout << "List: ";
    for (int n : lst) std::cout << n << " ";
    std::cout << std::endl;

    // 中间插入 O(1) (但需要先找位置 O(n))
    auto it = std::find(lst.begin(), lst.end(), 3);
    lst.insert(it, 99); // O(1) 插入在找到的位置后

    // splice 拼接 O(1)
    std::list<int> other = {10, 20, 30};
    lst.splice(lst.end(), other); // other 被清空

    std::cout << "After splice: ";
    for (int n : lst) std::cout << n << " ";
    std::cout << std::endl;

    // remove_if O(n)
    lst.remove_if([](int n) { return n % 2 == 0; });
    std::cout << "After remove_if (remove evens): ";
    for (int n : lst) std::cout << n << " ";
    std::cout << std::endl;

    // reverse O(n)
    lst.reverse();

    std::cout << "All list tests passed!" << std::endl;
    return 0;
}
```

14.7 面试高频问题

Q1: list 和 vector 如何选择？什么场景下 list 更好？

答案：绝大多数场景首选 vector。list 仅在以下极端场景更优：(1) 极大量元素频繁在中间插入/删除；(2) 需要 splice 语义将元素在容器间移动；(3) 绝对不能有迭代器失效。实际上，由于缓存局部性，即使 $O(n)$ 的 vector 插入也可能比 $O(1)$ 的 list 插入更快（元素数量不太大时）。

Q2: list 的 splice 操作是什么？时间复杂度？

答案：splice 将一个 list 的节点（或一段范围）"嫁接"到另一个 list 的指定位置，只修改指针不拷贝数据， $O(1)$ 。常用于实现 LRU cache 等场景。

Q3: forward_list 和 list 的区别？

答案：forward_list 是单向链表（只有 next 指针），内存更省但只能前向遍历，不支持 push_back（只有 push_front）。没有 size()（C++ 标准不要求，一些实现有但 $O(n)$ ）。

15. map 有序映射

15.1 是什么（定义）

std::map 是基于红黑树的有序关联容器，键值对按键排序，查找/插入/删除均为 $O(\log n)$ 。键唯一，自动排序。

```
std::map<std::string, int> m;
m["alice"] = 90;    // 插入或修改
m["bob"] = 85;
// 内部自动按 key 排序: alice < bob
```

15.2 为什么会出现

需要按键有序存储和快速查找（ $O(\log n)$ ）的键值对集合。支持范围查询（lower_bound/upper_bound）、按序遍历。

15.3 底层原理

map 的红黑树节点结构（简化）：

```
struct Node {
    std::pair<const Key, value> data;
    Node* left;
    Node* right;
    Node* parent;
    bool color; // RED or BLACK
};
```

红黑树保证：

1. 每个节点是红色或黑色
2. 根是黑色
3. 叶子（NIL）是黑色
4. 红色节点的子节点必须是黑色
5. 从根到叶的任何路径包含相同数量的黑色节点

→ 树的高度最多为 $2 \cdot \log(n+1)$ ，保证 $O(\log n)$ 操作

内存布局：每个节点额外 3 个指针 + 1 个颜色标记，通常在 32 字节开销（64位系统）。

15.4 优缺点

优点	缺点
自动按 key 排序	$O(\log n)$ 比 <code>unordered_map</code> 的 $O(1)$ 平均慢
支持范围查询	每个节点内存开销大
稳定的迭代器（插入不使其它迭代器失效）	缓存不友好（指针跳转）
可按序遍历	需要 key 支持 $<$ 运算符（或自定义比较器）

15.5 典型应用

```
// 1. 有序字典
std::map<std::string, int> scores;

// 2. 范围查询
auto lower = m.lower_bound("a");
auto upper = m.upper_bound("m");

// 3. 区间统计
auto range = m.equal_range(key);

// 4. 自定义排序
std::map<int, std::string, std::greater<int>> descending;
```

15.6 完整代码示例

```
#include <iostream>
#include <map>
#include <string>

int main() {
    std::map<std::string, int> scores;

    // 插入
    scores["alice"] = 90;
    scores["bob"] = 85;
    scores.insert({"charlie", 92});
    scores.emplace("dave", 78);

    // 查找
    if (auto it = scores.find("bob"); it != scores.end()) {
        std::cout << "bob's score: " << it->second << std::endl;
    }
}
```



```

// 遍历（按 key 排序）
std::cout << "All scores (sorted by name):" << std::endl;
for (const auto& [name, score] : scores) {
    std::cout << " " << name << ": " << score << std::endl;
}

// operator[] 的行为（不存在则插入默认值）
std::cout << "eve's score: " << scores["eve"] << std::endl; // 插入eve:0

// 范围查询
auto lb = scores.lower_bound("b");
auto ub = scores.upper_bound("d");
std::cout << "Names in [b, d):" << std::endl;
for (auto it = lb; it != ub; ++it) {
    std::cout << " " << it->first << std::endl;
}

// 删除
scores.erase("eve");

// 检查存在
if (scores.count("alice")) {
    std::cout << "alice exists" << std::endl;
}

std::cout << "All map tests passed!" << std::endl;
return 0;
}

```

15.7 面试高频问题

Q1: map 和 unordered_map 的区别？何时选哪个？

答案：map 基于红黑树， $O(\log n)$ ，有序，支持范围查询。unordered_map 基于哈希表，平均 $O(1)$ ，无序，最坏 $O(n)$ 。选 map：需要有序遍历或范围查询，或者 key 无法哈希。选 unordered_map：只需要 $O(1)$ 的快速查找，不关心顺序。

Q2: map 的 operator[] 和 insert 有什么区别？

答案：operator[] 如果 key 不存在会插入默认值并返回引用；如果存在返回现有值的引用。insert 如果 key 已存在则不插入（保留原值），返回 pair<iterator, bool> 指示是否插入成功。operator[] 不能用于 const map。

Q3: map 的 key 需要满足什么条件？

答案：必须支持 operator<（默认）或提供自定义比较器。比较器必须满足**严格弱序**（Strict Weak Ordering）。

16. unordered_map 无序映射

16.1 是什么（定义）

`std::unordered_map` 是基于**哈希表**的无序关联容器，查找/插入/删除平均 $O(1)$ ，最坏 $O(n)$ 。元素不按 key 排序，按哈希桶组织。

```
std::unordered_map<std::string, int> um;
um["alice"] = 90;
um["bob"] = 85;
// 不保证遍历顺序
```

16.2 为什么会出现

需要 $O(1)$ 平均查找的键值对存储，不关心顺序。哈希表比红黑树在大多数操作上更快。

16.3 底层原理

`unordered_map` 的哈希表结构（拉链法）：

```
bucket[0] -> Node -> Node -> Node （链表或开放寻址）
bucket[1] -> Node
bucket[2] -> （空）
...
bucket[N] -> Node -> Node
```

插入流程：

1. 计算 `hash(key)`
2. `hash % bucket_count()` 定位 `bucket`
3. 在 `bucket` 的链表中查找是否已存在（通过 `equal_to`）
4. 若不存在，插入到链表头部（或尾部）

扩容（`rehash`）：

当 `load_factor = size/bucket_count > max_load_factor` (默认1.0)

触发 `rehash`: `bucket_count` 扩大到下一个质数（通常2倍附近）

所有元素重新哈希分布到新 `bucket`

哈希碰撞处理：

标准库通常使用拉链法（`separate chaining`），每个`bucket`是一个链表。

16.4 优缺点

优点	缺点
平均 $O(1)$ 查找/插入/删除	最坏 $O(n)$ (大量碰撞或rehash)
不需要比较运算符	内存开销更大 (bucket数组+节点指针)
缓存性能优于 map (?)	遍历无序 (不能用于需要排序的场景)
自定义哈希函数灵活	rehash 导致迭代器失效

16.5 vs map 性能对比表

维度	map	unordered_map
底层结构	红黑树	哈希表
查找	$O(\log n)$	平均 $O(1)$, 最坏 $O(n)$
排序	自动有序	无顺序
内存占用	3指针+颜色	bucket数组+节点链表
范围查询	支持 lower/upper bound	不支持
迭代器稳定性	插入不失效	rehash后失效
key要求	< 运算符	hash函数 + == 运算符

16.6 完整代码示例

```
#include <iostream>
#include <unordered_map>
#include <string>

// 自定义哈希函数
struct PersonHash {
    std::size_t operator()(const std::pair<std::string, int>& p) const {
        return std::hash<std::string>()(p.first) ^
            std::hash<int>()(p.second);
    }
};

int main() {
    std::unordered_map<std::string, int> um;

    um["alice"] = 90;
    um["bob"] = 85;
    um.insert({"charlie", 92});

    // o(1) 平均查找
    if (auto it = um.find("alice"); it != um.end()) {
        std::cout << "alice: " << it->second << std::endl;
    }

    // 遍历（无序）
    std::cout << "All entries (unordered):" << std::endl;
    for (const auto& [k, v] : um) {
        std::cout << " " << k << ": " << v << std::endl;
    }

    // bucket 信息
    std::cout << "bucket_count: " << um.bucket_count() << std::endl;
    std::cout << "load_factor: " << um.load_factor() << std::endl;
```

```

std::cout << "max_load_factor: " << um.max_load_factor() << std::endl;

// rehash
um.reserve(1000); // 预分配至少1000个元素的容量
std::cout << "After reserve, bucket_count: "
          << um.bucket_count() << std::endl;

// 提取并修改 key (C++17)
auto nh = um.extract("alice");
if (nh) {
    nh.key() = "ALICE_UPPER";
    um.insert(std::move(nh));
}

std::cout << "All unordered_map tests passed!" << std::endl;
return 0;
}

```

16.7 面试高频问题

Q1: unordered_map 的哈希冲突如何解决？

答案：标准库通常使用拉链法（Separate Chaining），每个 bucket 维护一个链表，冲突的元素追加到链表上。当 load_factor 超过阈值时触发 rehash，扩大 bucket 数量使元素重新分布。

Q2: 如何为自定义类型实现 unordered_map 的 key？

答案：需要提供：(1) 哈希函数（std::hash 特化或自定义仿函数），(2) 相等比较函数（operator== 或自定义仿函数）。可以通过特化 std::hash 或在 unordered_map 的模板参数中指定。

Q3: unordered_map 的 rehash 和 reserve 有什么区别？

答案：rehash(n) 设置 bucket 数量至少为 n。reserve(n) 确保可以容纳至少 n 个元素而不触发 rehash（内部根据 max_load_factor 计算所需的 bucket 数量并调用 rehash）。

Q4: map 和 unordered_map 在遍历性能上有什么差异？

答案：map 的红黑树遍历是深度优先的，节点分散在堆上，缓存不友好。unordered_map 遍历所有 bucket 及其链表，也有很多指针跳转。两者遍历效率都不高。map 的优势在于可以只遍历一个范围，unordered_map 必须全遍历。

Q5: unordered_map 为什么没有 lower_bound / upper_bound？

答案：因为哈希表中元素没有顺序，不存在“小于某个 key 的所有元素”的概念。lower_bound/upper_bound 是有序数据结构（如红黑树）特有的操作。

17. set 有序集合

17.1 是什么（定义）

`std::set` 是基于**红黑树**的有序集合容器，元素唯一且自动排序。查找/插入/删除均为 $O(\log n)$ 。

```
std::set<int> s = {3, 1, 4, 1, 5, 9};  
// s 实际存储: {1, 3, 4, 5, 9} (自动排序, 去重)
```

17.2 核心特性

- 元素唯一且不可修改（const，修改会破坏排序）
- 基于红黑树，与 map 同构（可以理解为 key==value 的 map）
- 支持范围查询、有序遍历
- 插入返回 `pair<iterator, bool>`，bool 指示是否插入成功

17.3 代码示例

```
#include <iostream>  
#include <set>  
#include <algorithm>  
  
int main() {  
    std::set<int> s = {5, 2, 8, 2, 1, 8};  
    // 实际: {1, 2, 5, 8}  
  
    s.insert(3);  
    auto [it, inserted] = s.insert(3); // C++17 structured binding  
    if (!inserted) std::cout << "3 already exists" << std::endl;  
  
    s.erase(2);  
  
    // 范围查询  
    auto lb = s.lower_bound(3);  
    auto ub = s.upper_bound(7);  
    std::cout << "Elements in [3, 7]: ";  
    for (auto it = lb; it != ub; ++it)  
        std::cout << *it << " ";  
    std::cout << std::endl;  
  
    std::cout << "set size: " << s.size() << std::endl;  
    return 0;  
}
```

17.4 面试高频问题

Q1: set 和 map 的底层实现有何关系？

答案：两者都基于红黑树，本质是同一数据结构。set 可以看作 key 和 value 相同的 map (`set<T>` 约等于 `map<T, void>`)，但标准库为 set 做了优化（不需要存储额外的 value）。

Q2: multiset 和 set 的区别？

答案：`set` 不允许重复元素，`insert` 已存在元素会失败。`multiset` 允许重复元素，`insert` 总是成功。`multiset::erase(key)` 会删除所有等于 key 的元素。

18. unordered_set 无序集合

18.1 是什么（定义）

`std::unordered_set` 是基于**哈希表**的无序集合，元素唯一，查找/插入/删除平均 $O(1)$ 。用法与 set 类似，但元素无序。

18.2 与 set 对比

操作	set	unordered_set
查找	$O(\log n)$	平均 $O(1)$
排序	自动有序	无序
内存	较省	较大
范围查询	支持	不支持

18.3 代码示例

```
#include <iostream>
#include <unordered_set>

int main() {
    std::unordered_set<int> us = {5, 2, 8, 1, 9};

    us.insert(3);

    if (us.find(2) != us.end())
        std::cout << "2 found" << std::endl;

    std::cout << "Elements (unordered): ";
    for (int n : us) std::cout << n << " ";
    std::cout << std::endl;

    std::cout << "bucket_count: " << us.bucket_count() << std::endl;
    return 0;
}
```

19. string 字符串

19.1 是什么（定义）

`std::string` 是 C++ 标准库的字符串类，特化了 `std::basic_string<char>`。封装了动态字符数组，提供丰富的字符串操作接口。

```
std::string s = "hello";
std::string s2 = s + " world"; // "hello world"
```

19.2 为什么会出现

替代 C 风格字符串（`char*`），提供：(1) 自动内存管理；(2) 安全的操作接口；(3) 丰富的成员函数；(4) 与 STL 算法兼容。

19.3 底层原理

SSO (Small String Optimization, 短字符串优化)：

gcc/libstdc++ string 结构 (64位)：

```
struct string {
    char*    ptr_;           // 指向数据
    size_t   size_;          // 字符串长度
    union {
        char    buffer_[16]; // 本地缓冲区 (SSO)
        size_t   capacity_;   // 堆分配时的容量
    };
}; // 总共32字节
```

SSO工作原理：

- 字符串长度 <= 15 → 存储在内部`buffer_`（无堆分配）
- 字符串长度 > 15 → 堆分配，`ptr_`指向堆内存

SSO的优势：

- 短字符串无堆分配，极快
- 缓存友好

COW (Copy-on-write) 已被废弃：

C++11之前有些实现使用COW，但因线程安全性和标准委员会的要求（`operator[]/data()`不能复制），现代实现都不再用COW。

19.4 优缺点

优点	缺点
自动内存管理 (RAII)	内存占用较大 (32字节典型)
SSO优化 (短字符串快)	大量拼接可能有多次分配 (用reserve减少)
丰富的操作接口	隐式构造开销 (const char* 到 string)

优点	缺点
与STL算法兼容	Unicode支持不完善（需C++20 char8_t或ICU库）

19.5 常见操作性能表

操作	复杂度	说明
<code>s[i]</code>	$O(1)$	无边界检查
<code>s.at(i)</code>	$O(1)$	有边界检查（抛out_of_range）
<code>s + s2</code>	$O(n+m)$	创建新字符串
<code>s += s2</code>	$O(m)$	追加（均摊）
<code>s.find(t)</code>	$O(n*m)$	朴素匹配（不保证Boyer-Moore等）
<code>s.substr(pos, len)</code>	$O(len)$	C++11后拷贝（不再共享）
<code>s == s2</code>	$O(n)$	先比较长度

19.6 完整代码示例

```
#include <iostream>
#include <string>
#include <sstream>
#include <algorithm>

int main() {
    // === 构造 ===
    std::string s1 = "hello";
    std::string s2("world");
    std::string s3(5, 'x');           // "xxxxx"

    // === 拼接 ===
    std::string s4 = s1 + " " + s2;
    std::cout << s4 << std::endl;

    // === 查找 ===
    size_t pos = s4.find("world");
    if (pos != std::string::npos)
        std::cout << "'world' found at: " << pos << std::endl;

    // === 子串 ===
    std::string sub = s4.substr(0, 5); // "hello"
    std::cout << "substr: " << sub << std::endl;

    // === 比较 ===
    if (s1 == "hello") std::cout << "s1 equals hello" << std::endl;

    // === 转换 ===
```



```

int num = std::stoi("12345");
double d = std::stod("3.14159");
std::string numStr = std::to_string(42);
std::cout << "stoi: " << num << ", stod: " << d
          << ", to_string: " << numStr << std::endl;

// === C API交互 ===
const char* cstr = s4.c_str(); // 返回null结尾的C字符串
printf("c_str: %s\n", cstr);

// === data() vs c_str() ===
// C++11前: c_str()保证null结尾, data()不保证
// C++11后: 两者等价, 都返回null结尾的C字符串

// === 容量操作 ===
std::string big;
big.reserve(1000); // 预分配
std::cout << "capacity after reserve: " << big.capacity() << std::endl;

// === 字符串流 ===
std::ostringstream oss;
oss << "Hello " << 42 << " World " << 3.14;
std::string formatted = oss.str();
std::cout << formatted << std::endl;

// === 常见算法 ===
std::string upper = "hello";
std::transform(upper.begin(), upper.end(), upper.begin(),
               [](unsigned char c) { return std::toupper(c); });
std::cout << "upper: " << upper << std::endl;

std::cout << "All string tests passed!" << std::endl;
return 0;
}

```

19.7 面试高频问题

Q1: 什么是 SSO? std::string 的典型内存布局?

答案: SSO (Small String Optimization) 是短字符串优化。当字符串长度在 15 (gcc) 或 22 (msvc) 字符以内时, 直接存储在 `std::string` 对象内部的缓冲区中, 不需要堆分配。对象大小通常为 32 字节 (gcc, 64 位)。

Q2: c_str() 和 data() 有什么区别?

答案: C++11 之后两者等价, 都返回以 null 结尾的 C 字符串。C++17 添加了 non-const 重载版本的 `data()`, 可以修改返回的字符数组 (但不能修改 null 终止符位置)。

Q3: string 的 COW (写时复制) 为什么被废弃?

答案: (1) 多线程环境下 COW 的引用计数需要原子操作, 反而增加了开销; (2) `operator[]` 和 `data()` 的非 const 版本需要"写时复制", 但如果有两个线程同时调用非 const `operator[]`, 存在竞态条件; (3) SSO 在实践中比 COW 更高效。

Q4: std::string 和 std::string_view 的区别?

答案: (1) `string` 拥有数据, `string_view` 只是非拥有型视图 (指针+长度); (2) `string_view` 不分配内存, 创建开销极低; (3) `string_view` 不保证 null 结尾; (4) `string_view` 在函数参数中常用以避免不必要的拷贝。注意: `string_view` 可能导致悬空 (引用的数据已被释放)。

Q5: string 的 Operator+ 和 append 在性能上有什么区别?

答案: `s1 + s2` 总是创建新的临时 string (分配新内存 + 拷贝两个操作数)。 `s1.append(s2)` 在 s1 有足够 capacity 时直接在原地追加 (无新分配)。循环中拼接大量字符串时, 用 `append` 或 `operator+=` 配合 `reserve` 更高效。

19.8 延伸问题

- Unicode 字符串在 C++ 中如何处理 (`std::wstring`、`char8_t`、`codecvt`) ?
- `std::stringstream` vs `std::format` (C++20) 的性能对比?
- 如何实现一个支持 SSO 的自定义 string 类?

20. 模板 (Templates)

20.1 是什么 (定义)

模板是 C++ 的**泛型编程**机制, 允许编写与类型无关的代码, 由编译器在编译期根据实际使用的类型**实例化**出具体版本。分为**函数模板**和**类模板**两种。

```
// 函数模板
template<typename T>
T max(T a, T b) { return a > b ? a : b; }

// 类模板
template<typename T>
class Stack {
    std::vector<T> elems;
public:
    void push(const T& e) { elems.push_back(e); }
};
```

20.2 为什么会出现

1. **消除重复代码**: 不用为 `int`、`double`、`string` 各写一份 `max` 函数。
2. **类型安全**: 编译器自动检查所有实例化是否正确, 宏做不到。
3. **编译期多态**: 泛型编程的静态多态 (编译期确定类型) 相比运行时多态 (virtual) 没有虚函数表的开销。
4. **库开发的基础**: STL 全部基于模板实现。

20.3 底层原理

模板的编译模型：

1. 两阶段查找（Two-Phase Lookup）：

阶段1（模板定义时）：检查不依赖模板参数的名称

阶段2（模板实例化时）：检查依赖模板参数的名称

2. 隐式实例化：

编译器在第一次使用模板时生成具体代码

Stack<int> s; // 编译器生成 Stack<int> 的完整类定义

3. 显式实例化：

template class Stack<int>; // 强制生成 Stack<int> 的代码

4. 模板代码必须在头文件中（通常）：

因为编译器需要在实例化时看到完整定义

（C++20 modules 可能改变这一点）

关键概念：

- 模板不是代码，是"代码生成器"

- 未使用的模板不产生任何目标代码

- 每个不同的模板参数组合生成一份独立代码

20.4 优缺点

优点	缺点
代码复用，类型安全	编译时间增长（大量实例化）
零运行时开销（编译期确定）	错误信息极长且难读（C++20 concept改善）
静态多态（比虚函数快）	代码膨胀（每个类型一份代码）
元编程能力	必须在头文件中（传统模型）

20.5 典型应用

```
// 1. 泛型容器
std::vector<int> vi;
std::vector<std::string> vs;

// 2. 泛型算法
std::sort(v.begin(), v.end()); // 自动推导元素类型

// 3. 类型萃取
template<typename T>
void process(T t) {
    if constexpr (std::is_integral_v<T>) { /* 整数路径 */ }
    else { /* 其他类型路径 */ }
}
```

```
// 4. CRTP (奇异递归模板模式)
template<typename Derived>
class Base {
    void interface() {
        static_cast<Derived*>(this)->impl();
    }
};

// 5. Policy-Based Design
template<typename T, typename AllocPolicy, typename LockPolicy>
class ThreadSafeContainer { /* ... */ };
```

20.6 完整代码示例

```
#include <iostream>
#include <vector>
#include <type_traits>

// === 函数模板 ===
template<typename T>
T max(T a, T b) {
    return a > b ? a : b;
}

// === 类模板 ===
template<typename T, size_t Capacity = 10>
class FixedBuffer {
    T data_[Capacity];
    size_t size_ = 0;
public:
    bool push(const T& val) {
        if (size_ < Capacity) { data_[size_++] = val; return true; }
        return false;
    }
    const T& operator[](size_t i) const { return data_[i]; }
    size_t size() const { return size_; }
};

// === 非类型模板参数 ===
template<typename T, int N>
constexpr int arraySize(T (&)[N]) { return N; }

// === 默认模板参数 ===
template<typename Key, typename Value, typename Compare = std::less<Key>>
class OrderedMap { /* ... */ };

// === 成员模板 ===
class Normalizer {
    double offset_;
public:
    template<typename T>
    double normalize(T val) const { return static_cast<double>(val) - offset_; }
```

```
};

// === CRTP 模式 ===
template<typename Derived>
class SingletonBase {
public:
    static Derived& instance() {
        static Derived inst;
        return inst;
    }
protected:
    SingletonBase() = default;
};

class MySingleton : public SingletonBase<MySingleton> {
    friend class SingletonBase<MySingleton>;
    MySingleton() = default;
};

int main() {
    // 函数模板实例化
    std::cout << "max(3, 5) = " << max(3, 5) << std::endl;
    std::cout << "max(3.14, 2.71) = " << max(3.14, 2.71) << std::endl;

    // 类模板实例化
    FixedBuffer<int, 5> buf;
    buf.push(10); buf.push(20); buf.push(30);
    std::cout << "FixedBuffer: ";
    for (size_t i = 0; i < buf.size(); ++i)
        std::cout << buf[i] << " ";
    std::cout << std::endl;

    // 非类型模板参数
    int arr[10];
    std::cout << "arr size: " << arraySize(arr) << std::endl;

    // CRTP单例
    auto& s = MySingleton::instance();

    std::cout << "All template tests passed!" << std::endl;
    return 0;
}
```

20.7 面试高频问题

Q1: 模板的声明和定义为什么通常要放在头文件中?

答案: 模板不是实际的代码，而是“代码生成指南”。编译器在实例化模板时需要看到完整的模板定义才能生成代码。如果定义在 .cpp 中，其他编译单元看不到定义就无法实例化。解决方案：(1) 全部放头文件；(2) 在 .cpp 中显式实例化所有需要的类型；(3) C++20 modules。

Q2: typename 和 class 在模板参数中有什么区别?

答案：在 `template<typename T>` 和 `template<class T>` 中，两者**完全等价**，没有任何区别。但在嵌套依赖类型名（如 `typename T::iterator it`）中，必须用 `typename` 告知编译器这是一个类型名而非静态成员。

Q3: 什么是模板的两阶段查找（Two-Phase Lookup）？

答案：阶段一在模板定义时：查找不依赖模板参数的名字（如全局函数、标准类型）。阶段二在模板实例化时：查找依赖模板参数的名字（如 `T::foo()`）。这解释了为什么某些错误在实例化时才被检测到。

Q4: 什么是模板的代码膨胀？如何避免？

答案：每种类型参数组合生成一份独立代码，导致可执行文件变大。避免方法：(1) 提取非类型依赖的代码到非模板基类；(2) 避免过度的 `template<T, U, V, W>` 参数化；(3) 只在确实需要泛型的地方使用模板。

Q5: 模板特化和偏特化有什么区别？

答案：全特化（explicit/complete specialization）为特定类型组合提供完全不同的实现：`template<> class Stack<bool>{...}`。偏特化（partial specialization）为某类类型组合（如指针类型）提供部分定制：`template<typename T> class Stack<T*>{...}`。函数模板不能偏特化（但可以重载）。

20.8 延伸问题

- C++20 concept 如何简化模板编程？
- SFINAE（Substitution Failure Is Not An Error）的原理和应用？
- 模板元编程与 constexpr 如何选择？

21. 模板特化 (Template Specialization)

21.1 是什么（定义）

模板特化是为**特定类型参数**提供不同于通用模板的实现。分为**全特化**（为具体类型定制）和**偏特化**（为部分类型模式定制）。

```
// 通用模板
template<typename T> struct Traits { static const char* name() { return "unknown"; } };

// 全特化：为 int 定制
template<> struct Traits<int> { static const char* name() { return "int"; } };

// 偏特化：为所有指针类型定制
template<typename T> struct Traits<T*> { static const char* name() { return "pointer"; } };
```

21.2 为什么会出现

为特定类型提供优化或不同的语义。例如：`vector<bool>` 使用位压缩存储；`unique_ptr<T[]>` 使用 `delete[]` 而非 `delete`；类型萃取（type traits）通过特化回答类型问题。

21.3 底层原理

全特化语法：

```
template<>           // 空模板参数列表
class MyClass<int, 42> { /* int和42的定制版本 */ };
```

编译器实例化选择规则：

1. 先尝试全特化（最匹配）
2. 再尝试偏特化
3. 最后使用通用模板

函数模板的"特化"陷阱：

函数模板不支持偏特化（标准规定）

但可以通过重载实现类似效果：

```
template<typename T> void f(T);           // 通用
template<typename T> void f(T*);          // 针对指针的"重载"
template<> void f<int>(int);               // 全特化（合法）
```

21.4 典型应用

```
// 1. std::hash 特化（为自定义类型提供哈希）
namespace std {
    template<>
    struct hash<MyType> {
        size_t operator()(const MyType& m) const { /* ... */ }
    };
}

// 2. vector<bool> 特化（位压缩）
std::vector<bool> vb; // 使用特化版本

// 3. type traits 全特化
template<typename T> struct is_void : std::false_type {};
template<> struct is_void<void> : std::true_type {};
```

21.5 完整代码示例

```
#include <iostream>
#include <string>

// 通用模板
template<typename T>
struct TypeInfo {
    static std::string name() { return "unknown"; }
    static constexpr bool isPointer = false;
};

// 全特化: int
template<>
struct TypeInfo<int> {
```

```

    static std::string name() { return "int"; }
    static constexpr bool isPointer = false;
};

// 全特化: double
template<>
struct TypeInfo<double> {
    static std::string name() { return "double"; }
    static constexpr bool isPointer = false;
};

// 偏特化: 指针类型
template<typename T>
struct TypeInfo<T*> {
    static std::string name() { return TypeInfo<T>::name() + "*"; }
    static constexpr bool isPointer = true;
};

// 偏特化: const 类型
template<typename T>
struct TypeInfo<const T> {
    static std::string name() { return "const " + TypeInfo<T>::name(); }
    static constexpr bool isPointer = TypeInfo<T>::isPointer;
};

int main() {
    std::cout << "int:      " << TypeInfo<int>::name() << std::endl;
    std::cout << "double:   " << TypeInfo<double>::name() << std::endl;
    std::cout << "int*:     " << TypeInfo<int*>::name()
               << " (isPtr=" << TypeInfo<int*>::isPointer << ")" << std::endl;
    std::cout << "const int: " << TypeInfo<const int>::name() << std::endl;
    std::cout << "const int*: " << TypeInfo<const int*>::name() << std::endl;
    std::cout << "string:    " << TypeInfo<std::string>::name() << std::endl;

    return 0;
}

```

21.6 面试高频问题

Q1: 函数模板可以偏特化吗？如何实现类似效果？

答案：不能（C++ 标准明确禁止）。替代方案：(1) 用函数重载：`template<typename T> void f(T*)` 针对指针类型；(2) 用类模板的偏特化包装（委托给特化的仿函数类）；(3) 用 `if constexpr` (C++17) 在函数内分支。

Q2: 全特化和偏特化在语法上如何区分？

答案：全特化：`template<> class X<int, double> {...}`（空尖括号+完全指定的模板参数）。偏特化：`template<typename T> class X<T, int> {...}`（尖括号中有未被指定的模板参数）。

Q3: 模板特化在什么时机被实例化？

答案：和普通模板一样，在首次使用时（隐式实例化）。如果同一个类型组合既有通用模板又有全特化，优先使用全特化。如果全特化的定义在第一次使用后才出现，则为格式错误的程序（ODR违规）。

22. 偏特化 (Partial Specialization)

22.1 是什么 (定义)

偏特化允许为模板参数的一部分或某种**模式**提供定制实现，而非针对完全具体的类型。

```
// 通用
template<typename T, typename U> class Pair { /* ... */ };

// 偏特化: 第二个参数固定为int
template<typename T> class Pair<T, int> { /* ... */ };

// 偏特化: 两个参数类型相同
template<typename T> class Pair<T, T> { /* ... */ };

// 偏特化: 指针类型
template<typename T, typename U> class Pair<T*, U*> { /* ... */ };
```

22.2 底层原理

编译器在实例化时的选择顺序:

1. 全特化优先
2. 偏特化之间根据"更特化"原则选择 (更具体的匹配优先)
3. 都不匹配用通用模板

22.3 典型应用

```
// 1. unique_ptr<T[]> 的偏特化 (数组版本用 delete[])
template<typename T, typename Deleter>
class unique_ptr<T[], Deleter> { /* 使用delete[] */ };

// 2. 类型萃取 (判断是否为数组)
template<typename T> struct is_array : false_type {};
template<typename T> struct is_array<T[]> : true_type {};
template<typename T, size_t N> struct is_array<T[N]> : true_type {};

// 3. remove_reference 的偏特化
template<typename T> struct remove_reference { using type = T; };
template<typename T> struct remove_reference<T&> { using type = T; };
template<typename T> struct remove_reference<T&&> { using type = T; };
```

22.4 面试高频问题

Q1: 偏特化和全特化在实例化时谁优先?

答案: 全特化始终优先于偏特化。多个偏特化都匹配时, 选择"最特化"的那个 (符合偏序规则的那个)。如果两个偏特化无法比较谁更特化, 编译器报歧义错误。

Q2: 偏特化可以用来实现编译期条件选择吗?

答案：可以。经典的 `enable_if` + 偏特化组合常用于 SFINAE 场景。C++17 的 `if constexpr` 在很多场景下可以替代偏特化，使代码更清晰。

23. 可变参数模板 (Variadic Templates)

23.1 是什么（定义）

C++11 引入的可变参数模板允许模板接受**任意数量**的模板参数，通过**参数包 (parameter pack)** 和**包展开 (pack expansion)** 机制处理。

```
template<typename... Args>
void print(Args... args) {
    (std::cout << ... << args); // C++17 折叠表达式
}
```

23.2 为什么会出现

1. **类型安全的可变参数：**替代 C 风格 `printf`（类型不安全）和过多的重载版本。
2. **实现 `emplace` 系列函数：**`emplace_back` 需要接受任意数量和类型的参数。
3. **实现 `tuple`：**`std::tuple` 需要存储任意数量任意类型的元素。
4. **实现完美的工厂函数：**`make_shared`、`make_unique` 等。

23.3 底层原理

参数包展开的基本模式：

`Args...` → 参数包的声明
`args...` → 参数包的展开

递归展开（C++11/14 的方式）：

```
template<typename T>
void print(T t) { std::cout << t; } // 终止条件

template<typename T, typename... Args>
void print(T first, Args... rest) {
    std::cout << first;
    print(rest...); // 递归展开
}
```

折叠表达式（C++17，更简洁）：

`(args + ...)` // 一元左折叠：(((a1+a2)+a3)+a4)
`(... + args)` // 一元右折叠：(a1+(a2+(a3+a4)))
`(args + ... op init)` // 二元折叠（有初始值）

`sizeof...(Args)` → 参数包中参数的数量（编译期常量）

23.4 优缺点

优点	缺点
任意参数数量和类型	语法复杂，学习曲线陡峭
类型安全（区别于 va_list）	编译时间较长
C++17 折叠表达式非常简洁	C++11/14 需要递归+终止条件
是 emplace/tuple 等基础	错误信息难以理解

23.5 典型应用

```
// 1. 打印任意参数
template<typename... Args>
void log(Args&&... args) {
    (std::cout << ... << std::forward<Args>(args));
}

// 2. 求和
template<typename... Args>
auto sum(Args... args) { return (args + ...); }

// 3. 创建 tuple
auto t = std::make_tuple(1, 2.0, "hello");

// 4. 完美转发工厂
template<typename T, typename... Args>
std::unique_ptr<T> make_unique(Args&&... args) {
    return std::unique_ptr<T>(new T(std::forward<Args>(args)...));
}
```

23.6 完整代码示例

```
#include <iostream>
#include <string>
#include <memory>
#include <utility>

// === C++17 折叠表达式：格式化输出 ===
template<typename... Args>
void print(Args&&... args) {
    (std::cout << ... << std::forward<Args>(args)) << std::endl;
}

// === 递归求和 (C++14) ===
template<typename T>
T sum(T v) { return v; }

template<typename T, typename... Args>
```

```

T sum(T first, Args... rest) {
    return first + sum(rest...);
}

// === C++17 折叠表达式求和 ===
template<typename... Args>
auto fold_sum(Args... args) {
    return (args + ...); // 一元左折叠
}

// === 可变参数工厂函数 ===
template<typename T, typename... Args>
std::unique_ptr<T> my_make(Args&&... args) {
    return std::unique_ptr<T>(new T(std::forward<Args>(args)...));
}

// === 类型安全的 printf 实现 ===
template<typename... Args>
void safe_printf(const char* fmt, Args&&... args) {
    // 实际实现会用 sizeof...(Args) 验证参数数量
    std::cout << "Format: " << fmt << ", args count: "
                << sizeof...(Args) << std::endl;
}

class Widget {
public:
    std::string name;
    int value;

    Widget(std::string n, int v) : name(n), value(v) {
        std::cout << "Widget(" << name << ", " << value << ") created" << std::endl;
    }
};

int main() {
    std::cout << "=== 折叠表达式 print ===" << std::endl;
    print("Hello", " ", "World", "!", " ", 42);
    // 输出: Hello, World! 42

    std::cout << "\n=== 求和 ===" << std::endl;
    std::cout << "sum(1,2,3,4,5) = " << sum(1, 2, 3, 4, 5) << std::endl;
    std::cout << "fold_sum(1,2,3,4,5) = " << fold_sum(1, 2, 3, 4, 5) << std::endl;

    std::cout << "\n=== 工厂函数 ===" << std::endl;
    auto w1 = my_make<Widget>("Alpha", 1);
    auto w2 = my_make<Widget>("Beta", 2);

    std::cout << "\n=== sizeof...(Args) ===" << std::endl;
    safe_printf("%d %s %f", 42, "hello", 3.14);

    std::cout << "\nAll variadic template tests passed!" << std::endl;
    return 0;
}

```

23.7 面试高频问题

Q1: 可变参数模板的递归展开如何终止?

答案: 需要一个非可变参数的"终止函数"作为基准情况。例如: 先定义 `void print(T t)` 处理单个参数, 再定义 `void print(T first, Args... rest)` 递归调用。当参数包展开到只剩一个参数时, 匹配终止版本。

Q2: C++17 的折叠表达式 (Fold Expression) 有哪几种形式?

答案: (1) 一元左折叠: `(... op pack) → (a1 op a2) op ... op an`; (2) 一元右折叠: `(pack op ...) → a1 op (a2 op ... op an)`; (3) 二元左折叠: `(init op ... op pack)`; (4) 二元右折叠: `(pack op ... op init)`。支持的操作符: `+ - * / % ^ & | = < > << >> += -=` 等 32 种。

Q3: `sizeof...(Args)` 是什么?

答案: 编译期运算符, 返回参数包中元素的数量。注意不是 `sizeof(Args)` 或 `sizeof(Args...)`, 而是 `sizeof...(Args)`, 三点在括号内。

Q4: 可变参数模板和 C 的 `va_list/printf` 有什么区别?

答案: (1) 类型安全: 模板编译期检查类型, `va_list` 不检查; (2) `va_list` 要求参数类型可传递; (3) 模板无运行时开销, `va_list` 有运行时解析开销; (4) 模板可以处理非平凡类型 (如 `std::string`), `va_list` 只能处理 POD 类型。

24. `std::thread` 线程

24.1 是什么 (定义)

`std::thread` 是 C++11 引入的标准线程类, 表示一个**独立的执行线程**。构造时传入可调用对象, 线程立即开始执行。线程对象是 **move-only** 的 (不可拷贝)。

```
#include <thread>
std::thread t([]() { std::cout << "Hello from thread!" << std::endl; });
t.join(); // 等待线程结束
```

24.2 为什么会出现

标准化多线程编程, 取代平台相关的 `pthread/Windows Thread` 等 API, 使 C++ 程序跨平台一致地使用多线程。

24.3 底层原理

`std::thread` 的生命周期:

- 构造 → 创建 OS 线程 + 执行可调用对象
- `join()` → 阻塞等待线程结束
- `detach()` → 分离线程 (线程独立运行, 不再受管控)
- 析构 → 如果线程是 `joinable` 的 (未 `join` 也未 `detach`), 调用 `std::terminate!`

线程本地存储 (`thread_local`):

- `thread_local int counter = 0; // 每个线程一份`
- 类似 `static`, 但生命周期 = 线程生命周期

硬件线程数:

- `std::thread::hardware_concurrency()` → CPU 核心数 (提示值)

资源模型:

main thread

+-----+

| thread t |---- owns ----->| OS thread | <-- native_handle()

+-----+

|

t.join() → 等 worker 结束

t.detach() → 释放所有权, worker 独立运行

~t() when joinable → std::terminate!

worker thread

+-----+

24.4 优缺点

优点	缺点
标准化, 跨平台	必须 join 或 detach (否则 terminate)
支持任意可调用对象	move-only, 不能拷贝
可配合所有同步原语	线程创建/销毁开销大 (考虑线程池)
RAII 友好 (配合 jthread)	异常可能在非预期线程抛出

24.5 典型应用

```
// 1. 基本用法
std::thread t(workFunction, arg1, arg2);
t.join();

// 2. 配合 lambda 捕获
int result;
std::thread t([&result]() { result = heavyCompute(); });
t.join();

// 3. 多线程并行计算
std::vector<std::thread> threads;
```

```

for (int i = 0; i < N; ++i)
    threads.emplace_back(worker, i);
for (auto& t : threads) t.join();

// 4. RAII 线程包装 (C++20有 std::jthread)
class ThreadGuard {
    std::thread& t_;
public:
    explicit ThreadGuard(std::thread& t) : t_(t) {}
    ~ThreadGuard() { if (t_.joinable()) t_.join(); }
};

```

24.6 完整代码示例

```

#include <iostream>
#include <thread>
#include <vector>
#include <mutex>
#include <chrono>

std::mutex cout_mtx; // 保护 cout 的互斥锁

void worker(int id, int iterations) {
    for (int i = 0; i < iterations; ++i) {
        std::lock_guard<std::mutex> lock(cout_mtx);
        std::cout << "Thread " << id << ": iteration " << i << std::endl;
    }
}

struct Functor {
    void operator()(int n) const {
        std::lock_guard<std::mutex> lock(cout_mtx);
        std::cout << "Functor called with " << n << std::endl;
    }
};

int main() {
    std::cout << "Hardware concurrency: "
              << std::thread::hardware_concurrency() << std::endl;

    // === 基本用法 ===
    std::thread t1(worker, 1, 3);
    t1.join(); // 等待完成
    std::cout << "t1 finished" << std::endl;

    // === Lambda ===
    std::thread t2([]() {
        std::lock_guard<std::mutex> lock(cout_mtx);
        std::cout << "Lambda thread" << std::endl;
    });
    t2.join();
}

```

```

// === 仿函数 ===
std::thread t3(Functor(), 42);
t3.join();

// === 多线程 ===
std::vector<std::thread> threads;
for (int i = 0; i < 4; ++i) {
    threads.emplace_back([i]() {
        std::lock_guard<std::mutex> lock(cout_mtx);
        std::cout << "Pool thread " << i << std::endl;
    });
}
for (auto& t : threads) {
    if (t.joinable()) t.join();
}

// === move 语义 ===
std::thread t4([]{ /* ... */ });
std::thread t5 = std::move(t4); // t4 变为 not joinable
// t4.join(); // 错误! t4 不再管理线程
t5.join();

std::cout << "\nAll thread tests passed!" << std::endl;
return 0;
}

```

24.7 面试高频问题

Q1: std::thread 对象析构时如果还没有 join/detach 会发生什么?

答案: `std::terminate()` 被调用, 程序终止。这是一个常见陷阱。必须确保 `join()` (等待完成) 或 `detach()` (放弃管理)。C++20 引入的 `std::jthread` 在析构时自动 join, 更安全。

Q2: std::thread 可以被拷贝吗?

答案: 不可以。`std::thread` 是 move-only 类型 (与 `unique_ptr` 类似), 代表对 OS 线程的独占所有权。移动后原线程对象变为"未关联任何线程"的状态 (not joinable)。

Q3: thread_local 变量有什么特点?

答案: (1) 每个线程拥有独立的变量副本; (2) 生命周期与线程相同 (线程创建时初始化, 线程退出时销毁); (3) 多线程访问不需要同步 (各自有副本)。

Q4: std::thread::hardware_concurrency() 的返回值可靠吗?

答案: 只能作为提示。某些平台可能返回 0 (无法检测)。使用时应做 fallback 处理 (如默认使用 4 或 8)。

25. std::mutex 互斥锁

25.1 是什么（定义）

`std::mutex` 是 C++11 引入的互斥锁，用于保护共享数据，防止多线程同时访问造成的竞态条件。提供 `lock()` 和 `unlock()` 操作。

```
std::mutex mtx;
mtx.lock();
// 临界区
mtx.unlock(); // 必须手动释放（易出错）

// 推荐使用RAII包装器
std::lock_guard<std::mutex> lock(mtx);
// 临界区
// 自动unlock
```

25.2 为什么会出现

标准化互斥同步原语。在多线程环境下，两个及以上线程同时访问共享数据且至少有一个是写操作时，必须使用互斥锁保证数据完整性。

25.3 底层原理

Mutex 的种类：

std::mutex

→ 标准互斥锁（不可重入，不可递归）

std::recursive_mutex

→ 可重入锁（同一线程可多次lock）

std::timed_mutex

→ 带超时的互斥锁（try_lock_for/try_lock_until）

std::shared_mutex(C++17)

→ 读写锁（共享/独占）

RAII 锁包装器：

std::lock_guard

→ 最简单，构造lock析构unlock，不可手动unlock

std::unique_lock

→ 灵活，可手动lock/unlock，可转移所有权，配合条件变量

std::scoped_lock(C++17)

→ 可同时锁多个mutex（避免死锁）

Mutex 的底层实现：

通常基于操作系统的futex（Linux）或SRWLock（Windows）

用户态快速路径（原子操作）+ 内核态慢速路径（futex_wait）

25.4 优缺点

优点	缺点
标准化，跨平台	性能开销（特别是竞争激烈时）
RAII 包装器自动释放	可能死锁
多种类型满足不同需求	粗粒度锁限制并行度
可配合条件变量	递归锁有额外开销

25.5 典型应用

```
// 1. 基本保护
std::mutex mtx;
int shared_data = 0;
void increment() {
    std::lock_guard<std::mutex> lock(mtx);
    ++shared_data;
}

// 2. 带超时的锁
std::timed_mutex tmtx;
if (tmtx.try_lock_for(std::chrono::milliseconds(100))) {
    std::lock_guard<std::timed_mutex> lock(tmtx, std::adopt_lock);
    // ...
}

// 3. 避免死锁：同时锁多个mutex
std::scoped_lock lock(mtx1, mtx2, mtx3); // C++17

// 4. 读写锁 (C++17)
std::shared_mutex rw_mtx;
// 读: std::shared_lock lock(rw_mtx);
// 写: std::unique_lock lock(rw_mtx);
```

25.6 完整代码示例

```
#include <iostream>
#include <thread>
#include <mutex>
#include <vector>
#include <chrono>

class ThreadSafeCounter {
    mutable std::mutex mtx_;
    int value_ = 0;
public:
    void increment() {
        std::lock_guard<std::mutex> lock(mtx_);
        ++value_;
    }

    int get() const {
        std::lock_guard<std::mutex> lock(mtx_);
        return value_;
    }
};

int main() {
    ThreadSafeCounter counter;
```

```

std::vector<std::thread> threads;
for (int i = 0; i < 10; ++i) {
    threads.emplace_back([&counter]() {
        for (int j = 0; j < 1000; ++j) {
            counter.increment();
        }
    });
}

for (auto& t : threads) t.join();

std::cout << "Final count: " << counter.get() << std::endl;
// 期望值: 10 * 1000 = 10000

// === 死锁演示 (应避免) ===
std::mutex m1, m2;
// 线程A: lock(m1); lock(m2);
// 线程B: lock(m2); lock(m1);
// → 死锁!
// 解决: 使用 std::scoped_lock(m1, m2) 或 std::lock(m1, m2)

std::cout << "All mutex tests passed!" << std::endl;
return 0;
}

```

25.7 面试高频问题

Q1: lock_guard 和 unique_lock 有什么区别?

答案: lock_guard 是轻量级 RAII 锁包装, 构造时 lock, 析构时 unlock, 不能手动 unlock, 不能转移。

unique_lock 更灵活: 可以延迟 lock、手动 unlock/lock、转移所有权、配合条件变量使用 (condition_variable::wait 需要 unique_lock)。开销方面 lock_guard 稍小。

Q2: 递归锁 (recursive_mutex) 有什么问题?

答案: 递归锁允许同一线程多次 lock。但通常递归锁是设计缺陷的标志——说明类封装有问题 (一个 public 方法调用了另一个 public 方法, 都加了锁)。递归锁有额外计数器开销, 且可能掩盖真正的竞态条件。优先考虑重构设计, 通过 private 非锁版本 + public 加锁版本的方式解决。

Q3: 如何避免死锁?

答案: (1) 始终以相同顺序获取多个锁; (2) 使用 std::lock() 或 std::scoped_lock() 同时获取多个锁; (3) 使用超时锁 (try_lock_for); (4) 避免嵌套锁; (5) 使用锁层级系统。圣典方案: 始终用 std::scoped_lock(m1, m2, m3...)。

Q4: std::call_once 是什么用途?

答案: 配合 std::once_flag 保证某个函数在多线程环境中只执行一次。经典用法是实现线程安全的单例模式和惰性初始化。比 double-checked locking 更简单、更安全。

26. std::condition_variable 条件变量

26.1 是什么（定义）

`std::condition_variable` 用于在线程间**等待和通知**，使得一个线程可以等待某个条件成立，另一个线程满足条件后通知等待者。必须配合 `std::mutex` 和 `std::unique_lock` 使用。

```
std::mutex mtx;
std::condition_variable cv;
bool ready = false;

// 等待线程
std::unique_lock<std::mutex> lock(mtx);
cv.wait(lock, []{ return ready; }); // 等待 ready 变为 true

// 通知线程
{
    std::lock_guard<std::mutex> lock(mtx);
    ready = true;
}
cv.notify_one(); // 或 notify_all()
```

26.2 为什么会出现

实现线程间的**同步协作**。例如：生产者-消费者模式、线程屏障、任务队列等待、异步操作完成通知等。单纯靠轮询（busy-waiting）浪费 CPU。

26.3 底层原理

`wait()` 的工作原理：

- 原子地：释放锁 + 阻塞当前线程（进入等待队列）
- 被 `notify` 唤醒后：重新获取锁
- 检查条件（如果是 `wait(lock, pred)` 形式）
- 如果条件不满足，回到步骤1（防止虚假唤醒）

虚假唤醒（Spurious wakeup）：

线程可能在没有任何 `notify` 的情况下被唤醒。
这是操作系统层面的行为（POSIX/windows 都允许）。
因此必须用带条件的 `wait(lock, predicate)` 形式！

`notify_one` vs `notify_all`：

`notify_one`：唤醒一个等待线程（不确定哪个）
`notify_all`：唤醒所有等待线程（如状态变化影响所有等待者）

26.4 优缺点

优点	缺点
避免忙等待，节省 CPU	必须配合 mutex + unique_lock

优点	缺点
标准化，跨平台	可能发生丢失唤醒 (lost wakeup)
支持超时等待	可能发生虚假唤醒
通知可以选择一个或全部	调试困难 (时序问题)

26.5 典型应用

```
// 1. 生产者-消费者模式
std::queue<Task> tasks;
std::condition_variable cv;
std::mutex mtx;
// 生产者: { lock; tasks.push(task); } cv.notify_one();
// 消费者: cv.wait(lock, []{ return !tasks.empty(); });

// 2. 线程屏障
std::condition_variable barrier;
int count = 0;
// wait: cv.wait(lock, [&]{ return count >= N; });
// signal: ++count; if(count == N) cv.notify_all();

// 3. 异步任务完成通知
bool done = false;
// worker: { done = true; } cv.notify_one();
// main: cv.wait(lock, []{ return done; });
```

26.6 完整代码示例

```
#include <iostream>
#include <thread>
#include <mutex>
#include <condition_variable>
#include <queue>
#include <chrono>

// === 生产者-消费者 ===
class TaskQueue {
    std::queue<int> queue_;
    std::mutex mtx_;
    std::condition_variable cv_;
    const size_t max_size_ = 5;

public:
    void produce(int item) {
        std::unique_lock<std::mutex> lock(mtx_);
        // 队列满则等待
        cv_.wait(lock, [this] { return queue_.size() < max_size_; });
        queue_.push(item);
        std::cout << "Produced: " << item
```

```

        << " (size=" << queue_.size() << ")" << std::endl;
    lock.unlock(); // 通知前释放锁可以，但非必须
    cv_.notify_one(); // 通知消费者
}

int consume() {
    std::unique_lock<std::mutex> lock(mtx_);
    // 队列空则等待
    cv_.wait(lock, [this] { return !queue_.empty(); });
    int item = queue_.front();
    queue_.pop();
    std::cout << "Consumed: " << item
        << " (size=" << queue_.size() << ")" << std::endl;
    lock.unlock();
    cv_.notify_one(); // 通知生产者（队列有空位）
    return item;
}

};

int main() {
    TaskQueue tq;

    // 生产者线程
    std::thread producer([&tq]() {
        for (int i = 1; i <= 10; ++i) {
            tq.produce(i);
            std::this_thread::sleep_for(std::chrono::milliseconds(50));
        }
    });

    // 消费者线程
    std::thread consumer([&tq]() {
        for (int i = 1; i <= 10; ++i) {
            tq.consume();
            std::this_thread::sleep_for(std::chrono::milliseconds(100));
        }
    });

    producer.join();
    consumer.join();

    std::cout << "All condition_variable tests passed!" << std::endl;
    return 0;
}

```

26.7 面试高频问题

Q1: 什么是虚假唤醒？如何处理？

答案：虚假唤醒（spurious wakeup）是指线程在没有任何 notify 的情况下被操作系统唤醒。因此不能只写 `cv.wait(lock)`，必须使用带条件的 `cv.wait(lock, predicate)` 形式，在被唤醒后重新检查条件。

Q2: wait 为什么要传入一个 unique_lock 而不是 lock_guard？

答案: `wait` 需要临时释放锁（让生产者线程能够获取锁并修改条件），等待期间锁是释放的，被唤醒后重新获取锁。`lock_guard` 不能手动 `unlock`，只有 `unique_lock` 支持灵活的上锁/解锁操作。

Q3: `notify_one` 和 `notify_all` 的区别？使用建议？

答案: `notify_one` 唤醒一个等待线程（通常用于生产者-消费者，一个物品只被一个消费者处理）。
`notify_all` 唤醒所有等待线程（通常用于状态变化影响所有等待者，如 `shutdown` 通知）。不确定时用 `notify_all` 更安全但效率稍低。

Q4: 如何用条件变量实现一个线程安全的阻塞队列？

答案: 用 `std::queue` + `std::mutex` + `std::condition_variable`。生产者在队列满时 `wait`，消费者在队列空时 `wait`。push 后 `notify consumer`，pop 后 `notify producer`。

27. `std::atomic` 原子操作

27.1 是什么（定义）

`std::atomic<T>` 提供**无锁的原子操作**，保证对变量的读取、写入、修改是原子的（不可分割），不会出现读到半个值的情况。通常用于简单的共享变量（计数器、标志位），避免重量级锁。

```
std::atomic<int> counter{0};
counter++;                // 原子递增
counter.fetch_add(1);     // 原子加，返回旧值
int v = counter.load();  // 原子读取
counter.store(10);        // 原子写入
```

27.2 为什么会出现

多线程环境下，即使是最简单的 `int` 递增也不是线程安全的（读-改-写不是原子的）。对于简单的共享变量，使用 `mutex` 太重了，`std::atomic` 使用 CPU 指令级别的原子操作，性能远高于 `mutex`。

27.3 底层原理

内存顺序（Memory Order）：

<code>memory_order_relaxed</code>	→ 只保证原子性，不保证顺序（计数器专用）
<code>memory_order_consume</code>	→ （不推荐使用）
<code>memory_order_acquire</code>	→ 读屏障：之后的操作不能重排到此之前
<code>memory_order_release</code>	→ 写屏障：之前的操作不能重排到此之后
<code>memory_order_acq_rel</code>	→ <code>acquire</code> + <code>release</code>
<code>memory_order_seq_cst</code>	→ 顺序一致性（默认，最安全最慢）

CPU 指令层面的实现：

x86/x64 上大多数 `atomic` 操作对应 `LOCK` 前缀指令：

<code>lock inc [addr]</code>	→ <code>atomic<int>::operator++</code>
<code>lock cmpxchg [addr]</code>	→ <code>compare_exchange</code>

无锁标志（`is_lock_free`）：

`std::atomic<int>::is_always_lock_free` → 在大多数平台为 `true`

基本类型（`int`，`ptr`，`etc.`）通常是无锁的

大类型（自定义 `struct`）可能需要内部锁

CAS (Compare-And-Swap) 操作:

```
bool compare_exchange_weak(T& expected, T desired);
bool compare_exchange_strong(T& expected, T desired);
如果当前值 == expected, 则设置为 desired, 返回 true
否则更新 expected = 当前值, 返回 false
weak版本可能"虚假失败", 需要在循环中使用
```

27.4 优缺点

优点	缺点
极低开销 (CPU指令级别)	只适用于简单操作 (计数器、标志)
无锁, 不会死锁	复杂数据结构需要 lock-free 编程 (极难)
不需要 RAII 锁管理	memory order 容易用错
可移植 (跨平台原子语义)	过多 atomic 访问导致缓存行颠簸 (cache line bouncing)

27.5 典型应用

```
// 1. 无锁计数器
std::atomic<long long> request_count{0};
void handleRequest() { request_count++; }

// 2. 自旋锁 (spinlock)
std::atomic_flag lock = ATOMIC_FLAG_INIT;
while (lock.test_and_set(std::memory_order_acquire)) {} // spin
lock.clear(std::memory_order_release);

// 3. 无锁单例 (double-checked locking)
std::atomic<Singleton*> instance{nullptr};

// 4. 停止标志
std::atomic<bool> stop{false};
void worker() { while (!stop.load()) { dowork(); } }
```

27.6 完整代码示例

```
#include <iostream>
#include <thread>
#include <atomic>
#include <vector>
#include <cassert>

// === 无锁计数器 ===
std::atomic<long long> global_counter{0};

void increment_many(int n) {
    for (int i = 0; i < n; ++i) {
```



```

        global_counter++; // 原子操作
    }
}

// === 自旋锁实现 ===
class SpinLock {
    std::atomic_flag flag_ = ATOMIC_FLAG_INIT;
public:
    void lock() {
        while (flag_.test_and_set(std::memory_order_acquire)) {
            // spin (实际生产中应加 pause 指令)
        }
    }
    void unlock() {
        flag_.clear(std::memory_order_release);
    }
};

// === CAS 实现无锁栈的 push (简化版) ===
template<typename T>
class LockFreeStack {
    struct Node {
        T data;
        Node* next;
        Node(const T& d) : data(d), next(nullptr) {}
    };
    std::atomic<Node*> head_{nullptr};
public:
    void push(const T& val) {
        Node* node = new Node(val);
        node->next = head_.load(std::memory_order_relaxed);
        while (!head_.compare_exchange_weak(node->next, node,
            std::memory_order_release, std::memory_order_relaxed)) {
            // node->next 已被更新为最新的 head_
            // loop until CAS succeeds
        }
    }
    // pop, size 等略...
};

int main() {
    // === 原子计数器 ===
    const int num_threads = 4;
    const int per_thread = 10000;
    std::vector<std::thread> threads;

    for (int i = 0; i < num_threads; ++i) {
        threads.emplace_back(increment_many, per_thread);
    }
    for (auto& t : threads) t.join();

    std::cout << "Counter: " << global_counter.load()
        << " (expected " << num_threads * per_thread << ")"

```

```

        << std::endl;

// === 原子标志 ===
std::atomic<bool> ready{false};
std::thread worker([&ready]() {
    while (!ready.load(std::memory_order_acquire)) {
        std::this_thread::yield();
    }
    std::cout << "Worker detected ready flag!" << std::endl;
});
std::this_thread::sleep_for(std::chrono::milliseconds(100));
ready.store(true, std::memory_order_release);
worker.join();

// === is_lock_free ===
std::cout << "atomic<int> is lock free: "
        << std::atomic<int>::is_always_lock_free << std::endl;

std::cout << "All atomic tests passed!" << std::endl;
return 0;
}

```

27.7 面试高频问题

Q1: atomic 和 mutex 的区别？什么时候用 atomic？

答案：`atomic` 是无锁的（基于 CPU 原子指令），适用于简单操作（计数器、标志位、指针交换等），性能极高。`mutex` 是重量级锁，适合保护复杂临界区（多步骤操作、数据结构修改等）。原子操作不会死锁但只支持有限的类型和操作。

Q2: 什么是内存顺序（memory order）？默认的是什么？

答案：内存顺序控制原子操作前后的指令重排规则。默认是 `memory_order_seq_cst`（顺序一致性，最强保证，最安全但可能稍慢）。更弱的有 `relaxed`（只保证原子性，适合计数器）、`acquire/release`（配对使用，适合生产者-消费者）。

Q3: compare_exchange_weak 和 compare_exchange_strong 的区别？

答案：`weak` 版本可能出现虚假失败（即使值等于 `expected` 也返回 `false`），但性能更好（在某些平台上映射为一条指令）。`weak` 必须用在循环中（`while loop`）。`strong` 保证只在值真的不等于 `expected` 时才失败。一般用 `weak` + 循环更高效。

Q4: std::atomic_flag 是什么？有什么用？

答案：最简单的原子类型，只支持 `test_and_set()` 和 `clear()` 两个操作。保证是无锁的（`is_always_lock_free` 为 `true`）。主要用于实现自旋锁等低级同步原语。

28. std::future 异步结果

28.1 是什么（定义）

`std::future<T>` 表示一个**将来可用的值**——异步操作的结果。可以从 `std::async`、`std::packaged_task`、`std::promise` 获取。通过 `get()` 等待并获取结果（只能调用一次）。

```
std::future<int> fut = std::async(std::launch::async, []{ return 42; });
int result = fut.get(); // 阻塞等待，获取结果
```

28.2 为什么会出现

提供标准化的异步编程模型，不需要手动管理线程和同步。解决了"启动一个任务，稍后获取其返回值"的需求。

28.3 底层原理

future/promise 通信模型：

+-----+
| promise |
| (写端) |
+-----+

共享状态
set_value()
get()

+-----+
| future |
| (读端) |
+-----+

共享状态（shared state）：

- 存储结果值或异常
- 同步机制（条件变量/原子变量）
- 引用计数（决定何时释放）
- 存储在堆上

future 的特点：

- move-only（类似 unique_ptr）
- get() 只能调用一次（之后 future 变为 invalid）
- shared_future 可以多次 get()（可拷贝）

28.4 优缺点

优点	缺点
标准化异步结果传递	get() 只能调用一次
支持异常传递	没有回调机制（C++20 有协程改善）
可配合 async/promise/packaged_task	不支持 .then() 链式操作（有提案）
类型安全	shared_future 有额外开销

28.5 典型应用

```
// 1. 异步计算
auto fut = std::async(std::launch::async, heavyCompute, args);
// 做其他事情...
auto result = fut.get();

// 2. Promise-Future 手动配对
std::promise<int> prom;
auto fut = prom.get_future();
std::thread([&prom]{ prom.set_value(42); }).detach();
auto val = fut.get();

// 3. packaged_task
std::packaged_task<int(int)> task([](int x) { return x * 2; });
auto fut = task.get_future();
task(21); // 执行
auto val = fut.get(); // 42

// 4. shared_future
std::shared_future<int> sf = std::move(fut); // 可多次get
```

28.6 完整代码示例

```
#include <iostream>
#include <future>
#include <thread>
#include <chrono>
#include <vector>

int slow_compute(int n) {
    std::this_thread::sleep_for(std::chrono::milliseconds(100));
    return n * n;
}

int main() {
    // === std::async ===
    std::future<int> fut1 = std::async(std::launch::async, slow_compute, 10);

    std::cout << "Doing other work while computing..." << std::endl;

    int result1 = fut1.get(); // 阻塞等待结果
    std::cout << "Result: " << result1 << std::endl;

    // === std::promise ===
    std::promise<std::string> prom;
    std::future<std::string> fut2 = prom.get_future();

    std::thread producer([&prom]() {
        std::this_thread::sleep_for(std::chrono::milliseconds(200));
        prom.set_value("Hello from promise!");
    });
```

```

        // prom.set_value 只能调用一次
    });

    std::cout << "waiting for promise..." << std::endl;
    std::cout << "Got: " << fut2.get() << std::endl;
    producer.join();

    // === 等待多个 future ===
    std::vector<std::future<int>> futures;
    for (int i = 1; i <= 5; ++i) {
        futures.push_back(std::async(std::launch::async, slow_compute, i));
    }

    int sum = 0;
    for (auto& f : futures) {
        sum += f.get();
    }
    std::cout << "Sum of squares 1..5 = " << sum << std::endl;

    // === wait_for 非阻塞等待 ===
    auto fut3 = std::async(std::launch::async, []() {
        std::this_thread::sleep_for(std::chrono::seconds(2));
        return 42;
    });

    auto status = fut3.wait_for(std::chrono::milliseconds(100));
    if (status == std::future_status::timeout) {
        std::cout << "Not ready yet!" << std::endl;
    }
    // fut3 会在 main 结束前析构 (blocking get 或 detach)

    std::cout << "All future tests passed!" << std::endl;
    return 0;
}

```

28.7 面试高频问题

Q1: future 和 promise 的关系？

答案：promise 是写入端，future 是读取端。通过 promise.get_future() 关联。promise::set_value() 设置值，future::get() 获取值。它们共享一个内部的“共享状态” (shared state)。

Q2: future::get() 可以调用多次吗？

答案：不能。future::get() 会移动走存储的值，之后 future 变为 invalid。如果需要多次获取或广播给多个消费者，应使用 std::shared_future。

Q3: future::wait() 和 future::get() 的区别？

答案：wait() 只阻塞等待结果的可用性，不获取结果（可以多次调用）。get() 等待并获取结果（只能调用一次），获取后 future 变为 invalid。

Q4: std::promise::set_exception 有什么作用？

答案：向 future 传递异常而非正常值。调用后，future 的 `get()` 会重新抛出该异常。用于在线程间传递异步任务中的异常信息。

29. std::async 异步任务

29.1 是什么（定义）

`std::async` 是 C++11 引入的**高级异步工具**，启动一个异步任务并返回 `std::future`。可以指定启动策略：立即在新线程执行（`std::launch::async`）或延迟执行（`std::launch::deferred`）。

```
auto fut = std::async(std::launch::async, function, args...);
// fut 析构时会阻塞等待任务完成（如果使用 async 策略）
auto result = fut.get();
```

29.2 为什么会出现

提供比 `std::thread` 更高层次的异步编程抽象，自动管理线程和返回值传递，不需要手动 join/detach。

29.3 底层原理

启动策略：

`std::launch::async` → 在新线程中立即执行
`std::launch::deferred` → 延迟执行（在 `get()/wait()` 时才执行，且在调用者的线程）
`std::launch::async` | `std::launch::deferred`（默认）→ 由实现决定

关键陷阱：

```
auto fut = std::async(std::launch::async, f);
// 如果 fut 是临时对象（没保存），会在语句结束时析构
// async 策略下，future 的析构会阻塞等待任务完成！
// → 失去了异步的意义
```

正确用法：

1. 保存返回值：`auto fut = std::async(...);`
2. 或使用 `shared_future`

vs `std::thread` 的选择：

`thread` → 需要精细控制线程生命周期
`async` → 只关心计算结果
`async` → 自动管理线程（某种程度上是“线程池”）

29.4 优缺点

优点	缺点
使用简单，高层抽象	默认启动策略由实现决定（不确定因素）
自动线程管理和异常传播	async 策略下 future 析构会阻塞
返回 future 便于异步编程	没有内置线程池（每次创建新线程）

优点	缺点
支持延迟执行	不适合大规模并发任务（线程创建开销）

29.5 典型应用

```
// 1. 并行计算
auto f1 = std::async(std::launch::async, compute, data1);
auto f2 = std::async(std::launch::async, compute, data2);
auto result = f1.get() + f2.get();

// 2. 异步 I/O
auto fut = std::async(std::launch::async, []{
    return fetchFromNetwork(url);
});
// 做其他事情...
processData(fut.get());

// 3. 延迟计算 (lazy evaluation)
auto fut = std::async(std::launch::deferred, expensiveOp);
// 如果最终不需要结果, expensiveOp 不会执行
if (needResult) { auto r = fut.get(); }
```

29.6 完整代码示例

```
#include <iostream>
#include <future>
#include <vector>
#include <numeric>
#include <chrono>

// 并行求和 (分治)
long long parallel_sum(const std::vector<int>& v, size_t begin, size_t end) {
    size_t mid = begin + (end - begin) / 2;
    size_t size = end - begin;

    if (size < 1000) {
        return std::accumulate(v.begin() + begin, v.begin() + end, 0LL);
    }

    auto fut = std::async(std::launch::async,
        [&v](size_t b, size_t e) { return parallel_sum(v, b, e); },
        begin, mid);

    long long right = parallel_sum(v, mid, end);
    long long left = fut.get();

    return left + right;
}

int main() {
```

```

// === async 并行计算 ===
auto fut1 = std::async(std::launch::async, []() {
    std::this_thread::sleep_for(std::chrono::milliseconds(100));
    return 10;
});
auto fut2 = std::async(std::launch::async, []() {
    std::this_thread::sleep_for(std::chrono::milliseconds(100));
    return 20;
});

std::cout << "Parallel results: " << fut1.get() + fut2.get() << std::endl;

// === deferred 延迟执行 ===
auto fut3 = std::async(std::launch::deferred, []() {
    std::cout << "Deferred task executing now!" << std::endl;
    return 42;
});
std::cout << "Before get()..." << std::endl;
std::cout << "Result: " << fut3.get() << std::endl;
// 延迟任务在 get() 调用时才在调用者线程中执行

// === 并行分治求和 ===
std::vector<int> data(10000);
std::iota(data.begin(), data.end(), 1);

auto start = std::chrono::steady_clock::now();
long long sum = parallel_sum(data, 0, data.size());
auto end = std::chrono::steady_clock::now();

std::cout << "Parallel sum: " << sum << " (expected: "
    << (10000LL * 10001 / 2) << ")" << std::endl;
std::cout << "Time: "
    << std::chrono::duration_cast<std::chrono::milliseconds>(
        end - start).count()
    << " ms" << std::endl;

std::cout << "All async tests passed!" << std::endl;
return 0;
}

```

29.7 面试高频问题

Q1: std::async 和 std::thread 的区别？如何选择？

答案：(1) `async` 返回 `future` 便于获取返回值和异常；`thread` 无返回值。(2) `async` 自动管理线程生命周期（析构时阻塞或延迟执行）；`thread` 必须手动 `join/detach`。(3) `async` 支持延迟执行。选择：当只需要任务的结果时用 `async`；需要精细控制线程（优先级、亲和性等）时用 `thread`。

Q2: std::launch::async 和 std::launch::deferred 有什么区别？

答案：`async` 在新线程中立即执行（异步）。`deferred` 在 `get()/wait()` 首次调用时才在当前线程中执行（惰性求值，同步）。默认策略（两者组合）由实现决定，可能导致不确定性，建议显式指定策略。

Q3: `std::async` 返回的 `future` 析构时会阻塞吗？

答案：如果使用 `std::launch::async` 策略，`future` 的析构会阻塞等待任务完成。这是 C++ 标准的规定。因此必须保存 `async` 返回的 `future`（赋值给变量），否则临时 `future` 会立即析构，导致 `async` 变成同步调用。

Q4: `std::async` 适合做大规模并发吗？

答案：不太适合。每次 `std::async(std::launch::async, ...)` 会创建新线程，没有线程池机制。对于大量小任务，线程创建/销毁的开销可能超过任务本身。建议：(1) 自己实现线程池；(2) 使用 `std::execution::par` (C++17) 标准并行算法；(3) 使用第三方库（如 TBB）。

30. 章节总结

30.1 面试重点（按考察频率排序）

优先级	知识点	考察频次	典型问法
★★★★★	虚函数/多态	极高	vtable原理、纯虚函数、析构函数为何虚
★★★★★	智能指针	极高	unique/shared/weak 区别、循环引用
★★★★★	移动语义	极高	move/forward区别、RVO、noexcept
★★★★★	STL容器选型	极高	vector/list/map/unordered_map对比
★★★★	Lambda	高	底层实现、捕获方式、mutable
★★★★	RAII	高	原理、标准库RAII类、异常安全
★★★★	左值/右值	高	值类别判断、转发引用
★★★★	多线程	高	mutex/atomic、条件变量、async
★★★	auto/decltype	中	推导规则、区别
★★★	constexpr	中	const区别、编译期计算
★★★	模板	中	特化/偏特化、可变参数

30.2 高频考点速查表

考点	核心答案关键词
auto vs decltype(auto)	auto剥引用，decltype(auto)保留
nullptr vs NULL	类型安全，重载决议，模板推导
using vs typedef	模板别名，可读性
const vs constexpr	运行时vs编译期
lambda底层	匿名仿函数类，捕获=成员变量
std::move做了什么	只做类型转换，不动东西

考点	核心答案关键词
有名字的右值引用是左值	int&& r=42; r是左值
移动构造要noexcept	vector扩容回退到拷贝
不要return std::move	抑制RVO
unique_ptr vs shared_ptr	独占vs共享, 0vs16字节开销
vector扩容因子	2x(gcc)/1.5x(msvc)
map vs unordered_map	红黑树vs哈希表, O(logn)vs均摊O(1)
SSO	短字符串优化, <=15字符不堆分配
完美转发	forward+转发引用+引用折叠
thread析构	若joinable则terminate

30.3 常见陷阱

1. **auto 陷阱**: `auto x = expr;` 剥去引用, `auto x = {1,2,3};` 推导为 `initializer_list`
2. **decltype 陷阱**: `decltype(x)` 和 `decltype((x))` 结果不同 (无括号vs有括号)
3. **vector 陷阱**: 不满足标准容器要求, 返回代理对象, `auto b = v[0]` 存的是代理不是bool
4. **return std::move 陷阱**: 抑制 RVO/NRVO
5. **const 对象 move 陷阱**: `std::move(const_obj)` 实际调用的是拷贝构造函数 (`const T&&` 无法匹配 `T&&`)
6. **lambda 悬空引用**: 引用捕获的变量在 lambda 执行前被销毁
7. **shared_ptr 循环引用**: 导致内存泄漏
8. **线程未 join/detach**: 析构时 `std::terminate`
9. **条件变量虚假唤醒**: 必须用 `wait(lock, pred)` 形式
10. **async future 析构**: 临时 future 析构会阻塞

30.4 建议背诵内容

【必背1】值类别规则:

有名字的右值引用是左值!

`std::move` 返回 `xvalue`!

`decltype(变量名)` → 声明类型

`decltype((变量名))` → 左值引用

【必背2】移动语义的核心:

`std::move = static_cast<T&&>` (只转换, 不移动)

`std::forward` = 条件转换 (保留值类别)

返回局部变量不要写 `std::move` (依赖RVO)

移动构造函数必须加 `noexcept`

【必背3】Smart Pointer选择:

独占所有权 → `unique_ptr`

共享所有权 → `shared_ptr`

打破循环引用 → `weak_ptr`

能用`unique`就别用`shared`

【必背4】容器选择口诀：

默认选 `vector`

需要头尾插入 → `deque`

需要中间频繁插入/删除 → `list`（但其实`vector`通常更快）

需要有序+范围查询 → `map`

需要最快查找 → `unordered_map`

不允许重复 → `set` / `unordered_set`

【必背5】多线程要点：

简单计数 → `atomic`

临界区保护 → `mutex` + `lock_guard`

等待条件 → `condition_variable` + `unique_lock`

获取返回值 → `async` + `future`

不要忘记 `join/detach`

用 `scoped_lock` 防死锁

【必背6】Lambda 公式：

[捕获](参数) `mutable` -> 返回类型 { 体 }

[] → 不捕获 [=] → 值捕获全部 [&] → 引用捕获全部

捕获发生在定义时，不是调用时

30.5 学习路径建议

第一轮（基础扫盲）：

`auto` → `decltype` → `nullptr` → `using` → `constexpr`
`lvalue/rvalue` → `move` → `forward` → `RAII`

第二轮（核心攻坚）：

`lambda` → `smart pointers` → STL容器对比
`template` → `specialization` → `variadic`

第三轮（并发深入）：

`thread` → `mutex` → `condition_variable` → `atomic`
`future` → `async`

第四轮（综合提升）：

容器源码分析（`vector`扩容、`map`旋转、`hash rehash`）
智能指针实现
线程池 + 无锁队列
`CRTP` + `Policy-Based Design`

文档信息：

- 总字数：约 35,000 字
- 覆盖概念：29 个核心 C++ 知识点
- 完整代码示例：29+ 段（均可用 `g++ -std=c++17` 编译）

- 面试真题：150+ 道（含标准答案）
- 延伸问题：60+ 道

使用建议：建议将此文档作为面试前 2 周的突击复习材料，每天精读 4-5 个概念，配合手写代码练习。